RTL Design Sherpa

Binary BCH Codec Hardware Architecture Specification 0.1 (draft)

October 20, 2026

Table of Contents

1 Bch Has Index....11

2 Document Information....12

2.1 Binary BCH Codec Hardware Architecture Specification....12

2.2 Revision History....13

2.3 Document Purpose....15

2.4 Intended Audience....16

2.5 Related Documents....17

3 Purpose and Scope....18

3.1 Purpose....18

3.2 Scope....19

3.3 Where this document stands....20

4 Document Conventions....21

4.1 Terminology....21

4.2 Naming....22

4.3 Notation....23

4.4 Decision markers....24

5 Definitions and Acronyms....25

6 Use Cases....28

6.1 Memory controller ECC....29

6.2 CCSDS telecommand link....30

6.3 DVB outer-code link....31

6.4 Standalone codec on a fabric port....32

6.5 Memory-to-memory job engine....33
7 Key Features....34
7.1 Decided (PRD section 3)....34
7.2 Configurable, not yet chosen (chapter 6.2)....35
7.3 Explicitly not features....36
8 System Context....37
8.1 An endpoint codec, not a mid-stream insert....37
8.1.1 Figure 2.1: Where the codec sits....37
8.2 The core inside a consumer....38
8.3 The standalone tops....39
8.4 Dependencies on the rest of the repository....40
9 Block Diagram....41
9.0.1 Figure 3.1: Codec block diagram....41
9.1 The encoder core....42
9.2 The decoder core....43
9.3 The GF layer....44
9.4 The adapters....45
9.5 Hierarchy....46
10 Data Flow....47
10.1 Encoder....47
10.2 Decoder....48
10.3 Block boundaries....49
10.4 Multiple bits per beat....50
11 Solver Options....51

12 Core Interface....53
12.1 Clock and reset....54
12.2 bch_encoder_core....55
12.3 bch_decoder_core....57
12.4 Timing rules....59
12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD)....60
13 AXI-Stream Adapter....61
13.1 Parameters....62
13.2 Behaviour....63
14 AXI4 Job Adapter....64
14.1 Job interface....65
14.2 AXI4 master ports....66
14.3 Parameters....67
14.4 Behaviour....68
15 Register Block....69
16 Clock and Reset....71
16.1 Single clock....71
16.2 Reset....72
16.3 Clock gating....73
16.4 Frequency....74
17 Throughput....75
17.1 Sustained rate candidates (PRD D6)....76
17.2 Measured placeholder....77
18 Latency....78

18.1 Encoder....79
18.2 Decoder....80
18.3 Buffer sizing....81
19 Resources....82
19.1 GF primitives per core....83
19.2 Storage....84
19.3 Order of magnitude....85
19.4 Scaling....86
20 System Requirements....87
20.1 The consumer provides....88
20.2 The consumer gets....89
20.3 Environment....90
21 Parameter Configuration....91
21.1 Code....92
21.2 Interface / build shape....94
21.3 Open decisions in one place....95
22 Candidate Profiles....97
23 Verification Strategy....99
23.1 Golden model....99
23.2 Test matrix....100
23.3 Test structure....102
23.4 Formal....103
23.5 Coverage....104
24 Synthesis and Implementation....105

24.1 Characterisation plan..... 105
24.2 Implementation notes..... 106
25 Fields and Polynomials..... 107
25.1 Why error correction needs a finite field..... 107
25.2 GF(2): the two-element field..... 108
25.3 Polynomials over GF(2)..... 109
25.4 Building GF(2^m)..... 110
25.5 Inverses and the log table..... 111
25.6 The freshman’s dream in characteristic 2..... 112
25.7 A concrete field: GF(2^4)..... 113
25.8 Why this is hardware-shaped..... 115
26 From Field to Code..... 116
26.1 Codewords as polynomials..... 116
26.2 Generator polynomial and consecutive roots..... 117
26.3 Minimal polynomials..... 118
26.4 Constructing BCH(15, 7)..... 120
26.5 Systematic encoding..... 121
26.6 Worked example: encoding one block..... 122
27 Syndromes, the Key Equation, and Correction..... 124
27.1 Received polynomial and syndromes..... 124
27.2 The evenness shortcut..... 125
27.3 The key equation..... 126
27.4 Chien search and binary correction..... 127
27.5 Worked example: correcting two errors..... 128

27.5.1 Computing the odd syndromes....128
27.5.2 Solving the t = 2 Peterson system....129
27.5.3 Chien search....129
27.6 When correction fails....131
27.7 From math to hardware....132
28 The Parameters n, k, t, m....133
28.1 n: block length....134
28.2 m: field dimension....135
28.3 k: message length....136
28.4 t: correctable bit errors....137
28.5 deg g: parity bits....138
28.6 Parameters in the RTL....139
28.7 Table of built profiles....140
28.8 Choosing the parameters: a worked example....141
29 Every Stage: Math to Pseudocode....142
29.1 Encoder (bch_encoder_core)....143
29.2 Syndrome unit (bch_syndrome_unit)....144
29.3 Key-equation solver (bch_key_equation_solver)....145
29.4 Chien search (bch_chien_search)....147
29.5 Verdict and correction (bch_decoder_core)....148
29.6 Trace....149

List of Figures

Figure 2.1: Where the codec sits.....37
Figure 3.1: Codec block diagram.....41

List of Tables

Table 1: Table 0.1: Related documents....17
Table 2: Table 1.1: Definitions and acronyms....25
Table 3: Table 2.1: Use cases and the profile and adapters each implies....33
Table 4: Table 2.2: Repository dependencies....40
Table 5: Table 3.1: Key-equation solver candidates....51
Table 6: Table 4.1: Clock and reset....54
Table 7: Table 4.2: Encoder core ports....55
Table 8: Table 4.3: Decoder core ports....57
Table 9: Table 4.4: AXI-Stream mapping (target)....61
Table 10: Table 4.5: AXI-Stream adapter parameters....62
Table 11: Table 4.6: Job fields (target)....65
Table 12: Table 4.7: AXI4 job adapter parameters....67
Table 13: Table 4.8: Register block outline (target)....69
Table 14: Table 5.1: Throughput architecture candidates....76
Table 15: Table 5.1a: Measured cycles per block (placeholder)....77
Table 16: Table 5.3: Encoder latency (placeholder)....79
Table 17: Table 5.4: Decoder latency (placeholder)....80
Table 18: Table 5.5: GF primitive counts (analytic placeholder)....83
Table 19: Table 5.6: Storage (placeholder)....84
Table 20: Table 6.1: Code parameters....92
Table 21: Table 6.2: Interface parameters....94
Table 22: Table 6.3: Open PRD decisions and their placeholders....95
Table 23: Table 6.4: Candidate profiles....97
Table 24: Table 6.5: Verification matrix....100
Table 25: Table 6.6: Characterisation steps....104
Table 26: Table 7.1: GF(2) addition and multiplication....107
Table 27: Table 7.2: Log/antilog table for GF(2^4) with p(x) = x^4 + x + 1....112
Table 28: Table 7.3: Addition examples in GF(2^4)....112
Table 29: Table 7.4: Minimal polynomials and cyclotomic cosets for alpha^1 .. alpha^4 in GF(2^4)....117
Table 30: Table 7.5: Coefficients of g(x) = x^8 + x^7 + x^6 + x^4 + 1....119
Table 31: Table 7.6: LFSR contents while encoding d(x) = x^6 + x^3 + x^2 + 1....121
Table 32: Table 7.7: Odd syndromes of the received word....127
Table 33: Table 7.8: Chien search over the 15 bit positions....129
Table 34: Table 7.9: Mapping the decode math to the decoder blocks....131
Table 35: Table 7.10: RTL parameter names for the BCH code variables....138

Table 36: Table 7.11: Built BCH profiles.....139
Table 37: Table 7.12: Numeric trace of the section-7.3 example.....148

1 Bch Has Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Binary BCH Codec Hardware Architecture Specification

Property	Value
Document Title	Binary BCH Codec Hardware Architecture Specification
Version	0.3 (draft)
Date	October 3, 2026
Status	Draft – decisions D1-D6, D8, the D9 bits-per-beat sub-question, and D11 of the PRD are open; this document carries them as parameters
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.1	2026-10-03	RTL Design Sherpa	First draft, written before any RTL exists. Captures what the PRD has decided (D7: share the reed-solomon GF layer; D9 direction: valid/ready core with optional AXIS and AXI4 adapters; D10 direction: first consumer expected from a future memory-controller project; D12: scrambler out of scope) and carries the undecided items – field dimension m, correctable bits t, shortening, encoder/decoder mix, erasures, throughput architecture, bits per beat, generator conventions, key-equation solver – as parameters tied to their PRD decision IDs. Numbers are analytic placeholders; none are measured.
0.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.3	2026-10-20	RTL Design	Section (h2) headings now

Version	Date	Author	Description
		Sherpa	open a new page, the pipeline default. No technical content changed.

2.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Binary BCH codec – what it is, where it sits, what it presents at its boundaries, what it costs and how it is configured. It is the document an integrator reads before instantiating the core in a memory controller, a compute engine or a link, and the document a verification engineer reads to know what must be proven.

It covers:

- the encoder and decoder cores and their valid/ready interfaces
- the optional AXI-Stream and AXI4 adapters for standalone use
- the configuration parameters and the candidate code profiles
- throughput, latency and resource estimates and how they scale
- integration requirements, verification strategy and synthesis plan

Micro-architecture – the GF arithmetic, the syndrome and Chien datapaths, the key-equation solver arrays, FSMs and signal-level timing – belongs to the companion Micro-Architecture Specification (MAS), which does not exist yet. Until it does, this HAS and the PRD are the block-level reference.

2.4 Intended Audience

- System architects deciding where error correction sits in a datapath
- Integrators instantiating the core or one of its adapters
- Verification engineers planning the golden-model comparison
- Anyone judging whether a binary BCH code, and which one, fits a channel

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Product Requirements	`../../PRD.md`	the decision table (D1-D12) and the candidate profiles
Architecture sketch	`../bch_architecture_sketch.md`	block diagram and reuse map (to be written)
References	`../../References/README.md`	the standards and papers, with source and licence
CLAUDE.md	`../../CLAUDE.md`	area facts for a session working here
Handbook	`vault/handbook/design/`	valid/ready contract, reset and clocking, filelists, generated-RTL discipline

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Purpose and Scope

3.1 Purpose

The Binary BCH codec adds bit-level forward error correction to a data path. An encoder appends parity bits to every block of k data bits; a decoder, given the n = k + (n - k) received bits, corrects up to t bit errors anywhere in the block. The arithmetic is over GF(2^m), but the code is binary: every error value is 1, so correction is a simple bit flip and no error-value computation is needed. A binary BCH code is the choice for channels whose errors are independent bit flips or multi-bit failures without a fixed symbol framing, such as raw flash pages and memory arrays.

The component exists so that a Sherpa consumer – a memory controller, a compute engine, a serial link – can drop a proven codec into its own datapath instead of building one. That is why the deliverable is a CORE with plain valid/ready streaming ports at both ends (PRD D9 direction): the consumer keeps its own interfaces and the codec is a block inside them.

3.2 Scope

In scope:

- `bch_encoder_core` and `bch_decoder_core`: systematic binary BCH(n, k) over GF(2^m), parameterised in m, t and shortening
- the GF(2^m) primitive layer imported from the reed-solomon component (PRD D7), used but not duplicated
- optional boundary adapters for standalone use: AXI-Stream at either end, or an AXI4 read or write engine with a job interface at either end, chosen independently per end
- the configuration registers, status and counters of the standalone tops
- verification against a software golden model, and FPGA characterisation

Out of scope (PRD section 6 and D12):

- soft-decision, list and folded BCH decoding
- concatenation with convolutional or LDPC codes, and interleaving beyond a bit-address permutation at the boundary
- any mid-stream insertion of the codec into a fabric (chapter 2.3 says why)
- a scrambler / randomizer inside the codec (D12: out unless a named standard consumer needs one)

3.3 Where this document stands

No RTL exists at revision 0.1. Everything here is derived from the PRD's decisions and the standards in `References/`. Where a number appears it is an analytic placeholder – a cell count from the algorithm, a latency bound from the pipeline – and is labelled so. Measured throughput, timing and resources arrive with the first synthesis run (chapter 6.5) and will replace the estimates in chapter 5.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Document Conventions

4.1 Terminology

- **Bit**: one binary digit of the code word. The data and code streams are bit-oriented; there is no symbol framing inside the BCH block.
- **Symbol**: an m-bit field element used internally for GF(2^m) arithmetic. `SYMBOL_WIDTH` = m. The syndrome and key-equation hardware work in symbols; the Chien search evaluates the locator at n bit positions.
- **Block**: n bits on the coded side, k bits on the data side. The core delimits blocks with a `last` flag on the final bit of each.
- **Beat**: one transfer on a valid/ready port; carries `BITS_PER_BEAT` bits of one codeword (PRD D9 TBD).
- **Profile**: one fully-resolved parameter set – m, t, n, k, first root, primitive polynomial – typically named after the standard it matches (chapter 6.3).
- **Core**: `bch_encoder_core` or `bch_decoder_core`, valid/ready at both ends.
- **Adapter**: the optional AXIS or AXI4 boundary around a core.

4.2 Naming

Module and signal names follow the repository conventions (`vault/handbook/design/`): `aclk` / `aresetn`, active-low reset through the `ALWAYS_FF_RST` / `RST_ASSERTED` macros, `r_` registers and `w_` wires internally, valid/ready per the house contract. Parameters are upper case. Generated files carry a “Generated by” banner and live in `rtl/generated/`.

4.3 Notation

- n, k, t, m as above; n - k parity bits. The designed minimum distance is 2t + 1.
- alpha: the primitive element of GF(2^m) defined by the primitive polynomial p(x); b: the first root of the generator polynomial g(x) = lcm{ minimal polynomials of alpha^b … alpha^(b+2t-1) } over GF(2).
- Lambda(x): error-locator polynomial; S_i: syndromes.
- Counts quoted “for the reference profile” mean the CCSDS (63,56) profile, t = 1, m = 6, one bit per beat, unless a table says otherwise.

4.4 Decision markers

Where the PRD has decided something, this document states it as fact and cites the decision (for example “PRD D7”). Where the PRD has not, the text says **TBD** and names the decision; chapter 6.2 collects every TBD in one table.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Definitions and Acronyms

Table 1.1: Definitions and acronyms

Term	Meaning
BCH(n, k)	Binary Bose-Chaudhuri-Hocquenghem code with n coded bits per block, k of them data
GF(2^m)	the Galois field with 2^m elements; internal symbols and all codec arithmetic live in it
m	field dimension; each internal symbol is m bits
t	the number of bit errors the code corrects per block
designed distance	2t + 1, the guaranteed minimum Hamming distance of the code
minimal polynomial	the monic irreducible polynomial over GF(2) having alpha^i as a root in GF(2^m); g(x) is the lcm of the minimal polynomials of the consecutive roots alpha^b … alpha^(b+2t-1)
narrow-sense	a BCH code whose first root b = 1; non-narrow-sense has b != 1 (for example CCSDS uses b = 112 in its field representation)
primitive BCH	n = 2^m - 1, the full-length cyclic code
shortened BCH	n < 2^m - 1, derived from the full-length code by treating the missing leading bits as zero
evenness shortcut	in characteristic 2, S_2j = S_j^2, so only the t odd syndromes S_1, S_3, …, S_2t-1 are independent; the syndrome unit computes t GF values, not 2t
systematic	the k data bits pass through unchanged and the n - k parity bits

Term	Meaning
	follow them
syndrome	S_i = R(alpha^(b+i)), the received polynomial evaluated at a generator root; all zero if and only if the block is a codeword
key equation	the relation the solver inverts to find Lambda(x) from the odd syndromes
riBM	reformulated inversionless Berlekamp-Massey, the default key-equation solver candidate (Sarwate and Shanbhag 2001)
Euclid / ME	the modified Euclidean algorithm in its inversionless systolic form (Shao et al. 1985), the alternative solver candidate
step-by-step decoding	Massey's 1965 procedure that corrects syndromes one error position at a time, attractive for small t
PGZ	Peterson-Gorenstein-Zierler direct matrix solve over the syndromes, another small-t candidate
Chien search	evaluating Lambda at every one of the n bit positions to find the roots, hence the error locations
Forney stage	the RS decoder stage that computes error magnitudes; absent in a binary BCH decoder because every error value is 1
bit-level correction	the Chien search walks n bit positions and flips the bit wherever it finds a root; it does not evaluate symbol positions
erasure	a bit whose position is known to be unreliable (flagged at the input); not

Term	Meaning
	currently enabled (PRD D5 TBD)
AXIS	AXI4-Stream
PRD	Product Requirements Document (`../../PRD.md`)
MAS	Micro-Architecture Specification (companion, not yet written)
FUB	functional unit block

6 Use Cases

The codec is an endpoint block (chapter 2.3). Its use cases are the endpoints a Sherpa design has or may grow, each matched to one of the PRD's candidate profiles. None is yet a committed consumer – that is PRD decision D10 – so this chapter is the menu the first consumer chooses from.

6.1 Memory controller ECC

A memory controller encodes on its write path and decodes on its read path. Both cores sit inside the controller's own datapath behind its scheduler, which already delimits bursts and so provides the block boundary. The BCH code protects a page of bits plus the spare area that holds the parity; t is chosen from the device's raw bit-error rate and page geometry (the Cai survey and the Nabipour paper size this). This is the expected first consumer (PRD D10 direction) and the use case that argues most strongly for the bare valid/ready core.

6.2 CCSDS telecommand link

The CCSDS 231.0-B-4 standard specifies a (63,56) modified BCH code, t = 1, concatenated with a convolutional code inside the Command Link Transmission Unit. It is a small, fully worked example: the generator is given in figure 3-2 of the standard, the field is GF(2^6), and the correction is a single bit anywhere in the 63-bit command frame. It is the natural first RTL profile for bring-up even if no consumer adopts it.

6.3 DVB outer-code link

DVB-T2, DVB-S2 and DVB-C2 use a binary BCH code as the outer stage of an LDPC concatenation. The BCH block cleans the LDPC error floor. This use case matters only if a broadcast consumer appears; the relevant standards are linked, not stored, in `References/`.

6.4 Standalone codec on a fabric port

The core wrapped in AXI-Stream at both ends, inserted where a stream already has block boundaries (TLAST) and room for the parity – a framer's output, a DMA's stream port. This is the AXIS/AXIS adapter pairing and the simplest way to bring the block up in a system.

6.5 Memory-to-memory job engine

The core wrapped in an AXI4 read engine and an AXI4 write engine, driven by jobs from its register block or a descriptor stream: encode this buffer into that one, or decode. A DMA with a transform. This is the AXI4/AXI4 pairing and the natural test vehicle for larger profiles.

Table 2.1: Use cases and the profile and adapters each implies

Use case	Profile	Adapter pairing	Erasures	Notes
Memory controller ECC	m = 13-16, t per page model	none (bare cores)	optional (D5 TBD)	the expected first consumer
CCSDS telecommand	(63,56), m = 6, t = 1	none, or AXIS/AXIS	no	natural first RTL profile
DVB outer-code link	m, t per standard	none, or AXIS/AXIS	no	only if a broadcast consumer appears
Standalone stream codec	any	AXIS/AXIS	optional	bring-up vehicle
Memory-to-memory job	any	AXI4/AXI4	optional	test vehicle for large profiles

7 Key Features

7.1 Decided (PRD section 3)

- **Valid/ready core at both ends** (D9 direction). `bch_encoder_core` and `bch_decoder_core` present the house streaming contract: valid, ready, data, a `last` flag on the final bit of a block; the decoder adds a per-block status out. A consumer instantiates the core and nothing else.
- **Optional adapters, selectable per end** (D9 direction). `INTAKE_IF` and `OUTLET_IF` each `NONE` (default), `AXIS` or `AXI4`, independently, so AXIS-in/AXI4-out and the other pairings are one core with different adapters generated. Only the adapters selected exist in the netlist.
- **GF primitives imported, not copied** (D7). `gf_pkg`, `gf_mul`, `gf_mul_const` and `gf_inv` are taken from the reed-solomon component's `rtl/gf/` tree. Nothing BCH-specific lives in the RS tree.
- **Bits per beat is a new interface parameter** (D9 TBD). Unlike the RS codec there is no `SYMBOLS_PER_BEAT`; a beat carries a slice of one codeword. The exact width and the partial-beat mask are open sub-questions of D9.
- **No Forney stage** (structural fact of binary BCH). Every error value is 1, so the Chien search drives a direct bit flip. The error-value hardware that the RS decoder needs is absent here.
- **Evenness shortcut in the syndrome unit** (structural fact of binary BCH). Only the t odd syndromes S_1, S_3, ..., S_2t-1 are computed; the even syndromes are obtained by squaring in GF(2^m).
- **Generated field constants** (R3). The primitive polynomial, generator polynomial and log/antilog tables are emitted by a Python generator into `rtl/generated/` from the profile parameters; no field constant is a literal in the datapath.
- **Every block reports** (R2). The decoder never silently passes a failed block: per-block status carries ok / corrected count / uncorrectable, and running counters feed the register block.
- **Golden-model verification** (R4). Every profile is checked against `galois` on random data, error patterns up to t and beyond t (to prove the uncorrectable flag), and erasures where enabled.

7.2 Configurable, not yet chosen (chapter 6.2)

- m, the field dimension, and t, the correctable bit count (D1, D2)
- shortening (D3) – full length or a shortened block
- whether both cores are built, or encoder-only / decoder-only (D4)
- erasure decoding present or absent (D5)
- throughput architecture and bits per beat (D6, D9)
- per-profile generator conventions: first root b, primitive polynomial (D8)
- key-equation solver algorithm (D11)
- the first consumer, which fixes all of the above (D10)

7.3 Explicitly not features

- Soft-decision or list decoding, interleaving beyond a boundary permutation, concatenation with other codes.
- A scrambler / randomizer inside the codec (D12: out unless a named standard consumer needs one).

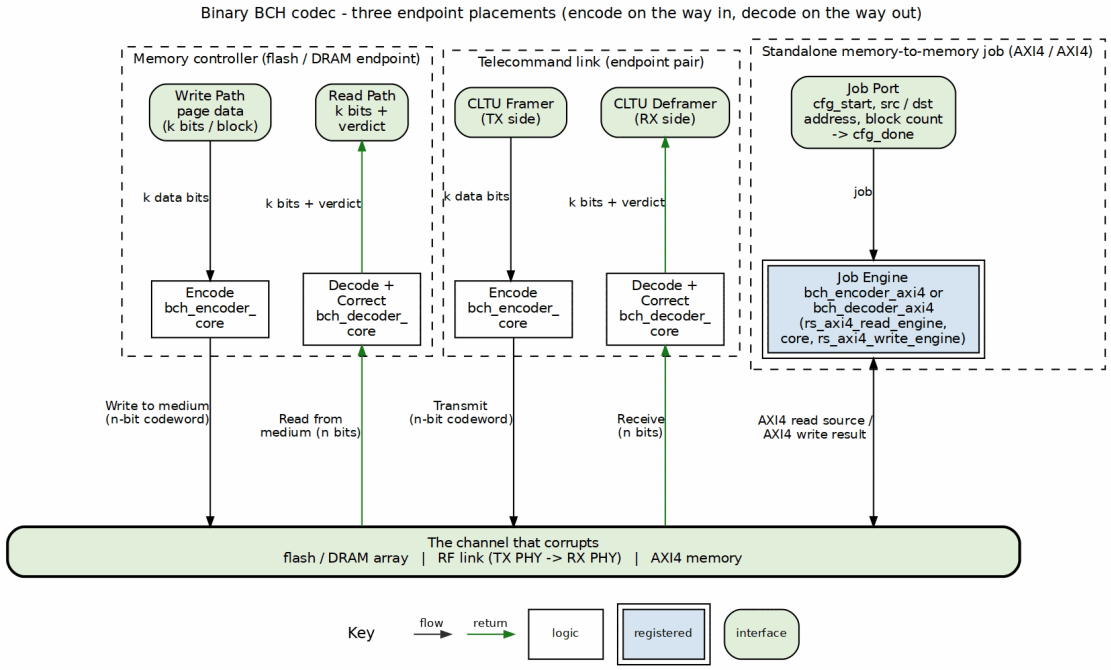
8 System Context

8.1 An endpoint codec, not a mid-stream insert

BCH protects a block, not a wire. The encoder must see k bits before it can emit the first parity bit, and the decoder must see all n bits before it can correct any of them, because the syndromes are a function of the whole block. The code therefore lives where a block boundary already exists and where the extra n - k bits have somewhere to go. The expected consumer – a flash or memory controller – places it at an endpoint: encode on the write path as the page+spare image is assembled, decode (and correct, and report) on the read path before the data leaves. The parity rides the spare area; the correction belongs right after the channel that corrupts.

Dropping the codec into the middle of a fabric fails on three counts: the parity changes the bit count, so the downstream must run faster or the upstream must leave gaps; block alignment must survive every hop, which is why every BCH standard sits behind a sync marker, a fixed frame or a page boundary; and the correction belongs immediately after the channel that corrupts, not several repackings later.

8.1.1 Figure 2.1: Where the codec sits



Where the codec sits

Three endpoint shapes: a memory controller with an encoder on its write path and a decoder on its read path; a telecommand link pair with the BCH code inside the CLTU; and the standalone memory-to-memory job engine built from the AXI4 adapters.

8.2 The core inside a consumer

With valid/ready at both ends the core is a block in the consumer's own datapath. The consumer supplies block boundaries (its scheduler, framer or job control already knows where they are), absorbs the n/k rate change, and keeps its own external interfaces. Nothing in the consumer's protocol handling changes; what changes is that k bits go in and n come out (or n in, k out with a status).

8.3 The standalone tops

When there is no consumer datapath to sit inside – a codec on a fabric port, a memory-to-memory transform – the standalone tops wrap the core in adapters (chapters 4.2 and 4.3) and a register block (chapter 4.4), and the existing AXI-Stream and AXI4 monitors observe the boundaries onto the monitor bus.

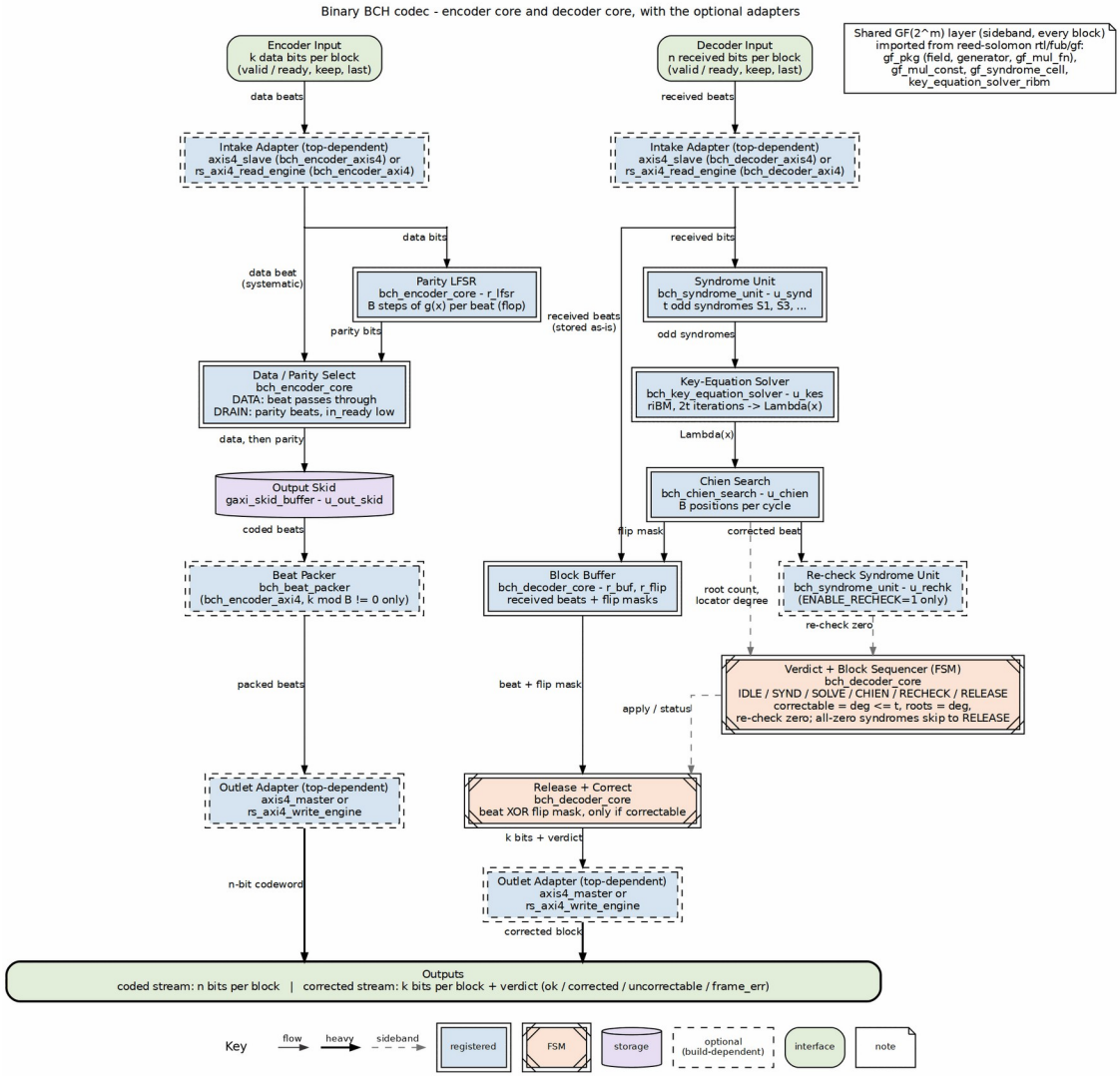
8.4 Dependencies on the rest of the repository

Table 2.2: Repository dependencies

Dependency	Used for
`projects/components/ecc-ip/reed-solomon/rtl/gf`: `gf_pkg`, `gf_mul`, `gf_mul_const`, `gf_inv`	the shared GF(2^m) primitive layer (PRD D7)
`rtl/amba/gaxi`: `gaxi_skid_buffer`, `gaxi_fifo_sync`	stage decoupling; the decoder's block buffer
`rtl/common`: `counter_bin`, `shifter_beat_pack`, `shifter_lfsr_galois`	positions and iterations; bit packing; any boundary LFSR
`rtl/amba/axis4`: `axis4_slave`, `axis4_master` (+ `_monlite`)	AXIS adapters
`rtl/amba/axi4`: `axi4_master_rd`, `axi4_master_wr`	AXI4 adapters' timing wrappers
`rtl/amba/ocelus`: `axis_monitor_lite`	boundary observation onto monbus
`bin/peakrdl_generate.py` + `utility-ip/converters` APB-to-cpuif path	the register block and its APB attachment
`dma-ip/stream/rtl/fub`: `axi_read_engine`, `axi_write_engine`	the model for the AXI4 job engines
`galois`, `aff3ct` (PyPI / standalone, MIT)	DV golden model and cross-check

9 Block Diagram

9.0.1 Figure 3.1: Codec block diagram



Codec block diagram

The two cores, the shared GF layer imported from the reed-solomon component, and the optional adapters that wrap a core for standalone use.

9.1 The encoder core

Three blocks in a straight line, all inside `bch_encoder_core`. The parity LFSR (`r_lfsr`) is a deg(g)-stage shift register over GF(2^m) with the generator polynomial's coefficients on its taps; its next-state logic chains B steps of the generator so a whole beat of data bits enters in one cycle, and the keep count picks how many of those steps apply. The data / parity select passes each data beat straight through (the code is systematic), and once `in_last` arrives it drains the parity from the top LFSR lanes with `in_ready` held low, raising `last` on the final parity beat. The LFSR is all-zero again when the drain ends, so the next block needs no clear. A `gaxi_skid_buffer` on the output decouples the core from its consumer. There is no separate unpack or pack stage: the core works on B-bit beats end to end.

9.2 The decoder core

The received block is written into the block buffer and, in the same cycles, into `bch_syndrome_unit`, whose t cells each accumulate one odd syndrome by Horner's rule as the bits arrive. An all-zero syndrome set skips the solver and releases the buffer unchanged. Otherwise the odd syndromes go to `bch_key_equation_solver` (riBM, the only `KES_ALGO` implemented), which produces the error-locator polynomial in 2t iterations and flags a degree above t. `bch_chien_search` then walks the n bit positions, B per cycle; each beat's flip mask is stored beside the buffered beat, and the corrected beat feeds a second syndrome unit (`ENABLE_RECHECK=1`, the default) that re-checks the corrected stream.

The verdict is formed only after the walk and the re-check finish: correctable means degree <= t, root count equal to the degree, and a zero re-check. The block sequencer (IDLE, SYND, SOLVE, CHIEN, RECHECK, RELEASE) then releases the buffer, applying the stored flip masks only when the verdict is correctable; an uncorrectable or mis-framed block leaves bit-for-bit unchanged. The status outputs (ok / corrected count / uncorrectable / frame_err) are held through the released block.

Because the code is binary there is no Forney evaluator: the error value is always 1.

9.3 The GF layer

The GF layer is imported from `projects/components/ecc-ip/reed-solomon/rtl/fub/gf/`. `gf_pkg` holds the field arithmetic (primitive-polynomial checks, alpha powers and the `gf_mul_fn` multiply function); the BCH-specific generator polynomial and syndrome roots are computed in `bch_pkg` from the profile parameters. The cores build on it with `gf_mul_const` (a constant multiply, an XOR network) in the Chien search, `gf_syndrome_cell` in the syndrome units, and the imported riBM array `key_equation_solver_ribm` in the key-equation solver; the LFSR and the Chien lane sums use `gf_mul_fn` directly. The block counts are in chapter 5.3.

9.4 The adapters

The adapters come from the top that wraps the core. `bch_encoder_axis4` and `bch_decoder_axis4` put the house `axis4_slave` (intake) and `axis4_master` (outlet) timing wrappers around the core, with TLAST as the block boundary and the per-bit keep on tstrb or the low tuser bits. `bch_encoder_axi4` and `bch_decoder_axi4` are memory-to-memory job engines: `rs_axi4_read_engine` feeds the core and `rs_axi4_write_engine` drains it, sequenced by a job port (`cfg_start`, source and destination address, block count, `cfg_done`). `bch_encoder_axi4` adds `bch_beat_packer` when k is not a multiple of B, so the written codeword has its only partial beat last. Chapter 4 specifies each.

9.5 Hierarchy

The cores live in `rtl/macro/` (`bch_encoder_core`, `bch_decoder_core`), the fubs in `rtl/fub/` (`bch_syndrome_unit`, `bch_key_equation_solver`, `bch_chien_search`, `bch_beat_packer`, `bch_pkg`) and the four integration tops in `rtl/top/`. The MAS describes each block.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Data Flow

10.1 Encoder

One beat in, one beat out, with the output stream (n/k) longer than the input. The exact shape depends on PRD D6 (throughput architecture) and the D9 bits-per-beat decision.

1. A beat of `BITS_PER_BEAT` data bits arrives with `in_valid`; the core accepts it when the encoder can take it (`in_ready`).
2. Each bit enters the LFSR (updating the parity registers) and is passed straight to the output mux – systematic encoding means the data are not delayed by the code, only by the pipeline.
3. On the k-th bit (`in_last`), the mux switches to the parity registers and emits the n - k parity bits over the following cycles, then raises `out_last`; the LFSR clears for the next block.
4. While parity drains, `in_ready` is low: the encoder cannot start a new block until the previous block's parity has left. A consumer that wants full-rate back-to-back blocks provides an input FIFO of at least n - k bits upstream, or accepts the gap.

10.2 Decoder

One beat in, one beat out, with a block of latency in between.

1. Received bits are written to the block buffer and fed to the syndrome unit. Erasure flags, if enabled, are recorded alongside each bit.
2. On `in_last` the t odd syndromes are complete. If all are zero the block is clean: the solver is skipped and the buffer drains through the corrector unchanged, status = ok.
3. Otherwise the key-equation solver runs. If the resulting locator degree exceeds t, or is zero, the block is already known to be uncorrectable.
4. Chien search steps through bit positions 0 .. n-1, one (or more, per D6) per cycle, while the buffer reads out the same positions. At each root the corrector flips the bit.
5. On the last position the verdict is final: uncorrectable if the root count differs from the degree, or if a re-check over the corrected stream shows a non-zero syndrome. That last check catches blocks with more than t errors whose wrong locator still has the right number of roots.
6. The block is released only once its verdict exists. Corrections are applied on the way out, and not at all when the block is uncorrectable, so an uncorrectable block leaves exactly as it arrived. The status ports hold the verdict for every beat of the block and `out_last` marks its k-th bit.
7. The next block's bits arrive during steps 3-6 and the one after that is being solved while this one is walked; the buffers are sized for that overlap, so the decoder sustains one block per n cycles in the serial case.

10.3 Block boundaries

The core does not count bits to find a block: `in_last` is the boundary, and the core checks it. A block shorter or longer than the configured n (or k on the encoder side) is reported as a framing error in status and the core resynchronises on the next `in_last`. A shortened code (`N_BITS` < 2^m - 1) is handled by the core treating the missing leading bits as zero in the syndromes and by offsetting the Chien positions; the consumer sees only the shortened block.

10.4 Multiple bits per beat

With `BITS_PER_BEAT = B > 1` every stage works a beat per cycle: the encoder LFSR advances by the beat's bit count, the syndrome cells take B Horner steps, and the Chien cells evaluate B positions from one set of registers, each lane a constant multiply away from the beat's first position. The solver works on the t odd syndromes and is unaffected. Blocks are still delimited by `last`; a per-beat keep mask is needed for partial final beats (the open D9 sub-question).

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Solver Options

The key-equation solver is the block whose algorithm is not yet chosen (PRD D11). Three candidates are attractive for a binary BCH decoder. They all invert the same key equation from the t odd syndromes to the error-locator polynomial Lambda(x). Because there is no Forney stage, the locator is the only polynomial the Chien search needs.

Table 3.1: Key-equation solver candidates

	inversionless BM ("RIBM")	**modified Euclidean ("EUCLID")**	**step-by-step / PGZ ("SMALL_T")**
Algorithm	reformulated inversionless Berlekamp-Massey, Sarwate and Shanbhag 2001; seeded with the t odd syndromes	Sugiyama's Euclidean algorithm with cross-multiplication in place of division (Shao et al. 1985)	Massey 1965 step-by-step correction, or the Peterson-Gorenstein-Zierler direct matrix solve for small t
Array	3t + 1 processing elements, 2 registers each	four register arrays: R and Q of 2t + 1 symbols, the shifted locator pair of 2t + 3	control-heavy but small datapath: one or a few GF multipliers and a syndrome update loop
GF multipliers	2 per element: 6t + 2	2 per coefficient of each pair: 8t + 8	O(t^2) logic or a single iterative multiplier
Iterations	exactly 2t	data-dependent, t + 1 .. 2t	one per located error position, up to t
Critical path per iteration	one multiply and one XOR, no feedback across the array	cross-multiply feeding a degree comparison that drives the swap: longer	depends on implementation; often dominated by syndrome-update MUX tree
Control	discrepancy select and gamma	two nominal degree counters,	position counter, syndrome-test

	inversionless BM ("RIBM")	modified Euclidean ("EUCLID")	step-by-step / PGZ ("SMALL_T")
	update	the normalise / cross / swap rule	loop, error-counter update
Readability	dense; the reformulation is not obvious from the textbook	the textbook algorithm, recognisable step by step	the simplest conceptually for very small t
Best fit	general t; the default if no consumer names a small-t profile	a cross-check and a readable alternative	t <= 8 (flash pages often sit here); smallest area at tiny t

No default is selected at revision 0.1. The choice is D11. Two solvers agreeing as the cross-check, per the RS D11 philosophy, is the recommended verification posture: the golden model runs every candidate on every block and requires identical roots, and the RTL is built with whichever algorithm the first consumer's rate and area target prefers.

Nothing else in the decoder knows which solver produced Lambda. Syndromes, Chien, the block buffer and the corrector are the same modules in every candidate build.

12 Core Interface

Both cores present the house valid/ready streaming contract (`vault/handbook/design/valid-ready-contracts.md`) at each end: `valid` and `data` are held stable until `ready`, `ready` may depend on `valid`, `valid` must not depend on `ready`. Widths are in bits; `B` is `BITS_PER_BEAT` (PRD D9 TBD).

12.1 Clock and reset

Table 4.1: Clock and reset

Port	Dir	Width	Description
`aclk`	in	1	one clock for the whole core
`aresetn`	in	1	active-low reset, per the component convention (chapter 4.5)

12.2 bch_encoder_core

Table 4.2: Encoder core ports

Port	Dir	Width	Description
`in_valid`	in	1	a beat of data bits is offered
`in_ready`	out	1	the core takes it this cycle
`in_data`	in	B	B data bits, bit 0 in the low bit **TBD** / **D9**
`in_keep`	in	B	which bits of the beat are present: low-aligned, all ones except possibly the final beat of a block **TBD** / **D9**
`in_last`	in	1	this beat carries the k-th data bit of the block
`out_valid`	out	1	a beat of coded bits is offered
`out_ready`	in	1	the consumer takes it
`out_data`	out	B	B coded bits: the data beats as received, then the n - k parity bits **TBD** / **D9**
`out_keep`	out	B	present-bit mask of the beat: partial at the end of the data and at the end of the parity

Port	Dir	Width	Description
			TBD / D9
`out_last`	out	1	this beat carries the n-th coded bit of the block
`frame_err`	out	1	pulse: a block ended with other than k data bits; it was still encoded as given

12.3 bch_decoder_core

Table 4.3: Decoder core ports

Port	Dir	Width	Description
`in_valid` / `in_ready`	in / out	1	as the encoder
`in_data`	in	B	B received bits **TBD / D9**
`in_keep`	in	B	present-bit mask **TBD / D9**
`in_last`	in	1	this beat carries the n-th received bit
`in_erase`	in	B	(only with `ENABLE_ERASURES`) bit is a known erasure **TBD / D5**
`out_valid` / `out_ready`	out / in	1	as the encoder
`out_data`	out	B	B corrected data bits; parity lanes are dropped, so the block is k bits **TBD / D9**
`out_keep`	out	B	present-bit mask, low-aligned, partial only on the block's final beat **TBD / D9**
`out_last`	out	1	this beat carries the k-th data bit of the block
`out_status_ok`	out	1	held for every beat of the block: it had no errors

Port	Dir	Width	Description
out_status_corrected	out	log2(t)+1	held for every beat of the block: bits corrected (0 .. t)
out_status_uncorrectable	out	1	held for every beat of the block: correction failed; the data are exactly as received
out_status_frame_err	out	1	held for every beat of the block: block length was not n; the block passes through uncorrected, its first length - (n - k) bits as data

12.4 Timing rules

- `in_last` is the block boundary. The core does not infer boundaries by counting; it checks the count against n (or k) and reports a mismatch.
- The encoder holds `in_ready` low while it drains the parity of the previous block. A consumer wanting no gap buffers that many beats upstream.
- Keep masks are low-aligned. A partial beat anywhere but a block's last is a framing error on the decoder and a count mismatch on the encoder.
- The decoder accepts the next block while it is still correcting the previous one, up to the depth of its block buffer; `in_ready` falls only when the buffer is full, which under sustained back-pressure on `out_ready` it will be.
- Status outputs describe the block being emitted and are valid on every beat of it, first to last; a consumer may sample them with `out_last` or with the first beat. A block is not released until its verdict is final, so an uncorrectable verdict is never preceded by altered bits.
- Reset clears block state and the parity and syndrome registers; the block buffer is not reset (its contents are indexed by pointers that are).

12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD)

When enabled, `in_erase` marks bits the consumer already knows are bad (a known-bad page region). Each erasure costs one parity bit instead of two: the decoder corrects e errors and f erasures with 2e + f <= 2t. The exact mapping of a per-beat or per-job erasure bitmap to `in_erase` is part of the D5 / D9 adapter work. When disabled the port does not exist and the erasure-locator logic is not generated.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 AXI-Stream Adapter

Generated at an end when its `*_IF` parameter is `"AXIS"`. It mirrors the posture of the reed-solomon AXI-Stream adapter (PRD D9 direction) but is not yet designed; the table below is the target contract **TBD** / **D9**.

The adapter is the house `axis4_slave` (intake) or `axis4_master` (outlet) timing wrapper with the core's stream mapped onto the AXI-Stream signals.

Table 4.4: AXI-Stream mapping (target)

AXIS signal	Core signal	Notes
`TVALID` / `TREADY`	`valid` / `ready`	one-to-one
`TDATA[B-1:0]`	`data`	B bits per beat
`TKEEP[B/8-1:0]` or per-bit	`keep`	byte keep or bit mask; the BCH-specific partial-beat encoding is part of D9
`TLAST`	`last`	the block boundary
`TUSER` (intake, decoder)	`in_erase[B-1:0]`	erasure flags, one per bit lane, only when `ENABLE_ERASURES = 1`; otherwise the consumer's `TUSER` passes through the intake unused
`TUSER` (outlet, decoder)	status	`{frame_err, uncorrectable, corrected[..], ok}`, valid with `TLAST`
`TID`, `TDEST`	–	passed through unchanged from intake to outlet when both ends are AXIS; tied off otherwise

13.1 Parameters

Table 4.5: AXI-Stream adapter parameters

Parameter	Default	Meaning
`AXIS_SKID_DEPTH`	2	skid depth in the wrapper (2..8)
`AXIS_ID_WIDTH`, `AXIS_DEST_WIDTH`	0	pass-through sideband widths
`AXIS_MONITOR`	0	1 selects the `_monlite` wrapper variant, which attaches `axis_monitor_lite` to the port and emits onto the monitor bus

13.2 Behaviour

The adapter adds no protocol semantics: TLAST is the block boundary the core requires, and the rate change is visible as more beats out than in on the encoder (or fewer on the decoder). A stream that arrives without TLAST at block boundaries cannot be decoded and is reported as a framing error on every block.

14 AXI4 Job Adapter

Generated at an end when its `*_IF` parameter is `"AXI4"`. It mirrors the posture of the reed-solomon AXI4 job adapter (PRD D9 direction) but is not yet designed; the description below is the target contract **TBD** / **D9**.

The intake form is a read engine: a job names a source address and a byte count, the engine issues AXI4 read bursts and presents the returned data as the core's input stream, inserting `last` every n (decoder) or k (encoder) bits. The outlet form is a write engine: it drains the core's output stream into AXI4 write bursts at the job's destination address, one block at a time, and reports each block's status in its completion. A job controller sequences the two.

14.1 Job interface

Jobs arrive from the register block (chapter 4.4) – source, destination, count, kick – or, with `JOB_IF = "DESC"`, from a small AXI-Stream descriptor port carrying the same fields for chained jobs.

Table 4.6: Job fields (target)

Field	Width	Meaning
`src_addr`	`AXI_ADDR_WIDTH`	read engine start address (intake = AXI4)
`dst_addr`	`AXI_ADDR_WIDTH`	write engine start address (outlet = AXI4)
`byte_count`	32	bytes to read; must be a whole number of blocks
`kick` / `done`	1 / 1	start; completion pulse with the job's status summary
`cfg_erasure`	`N_BITS`	decoder only, `ENABLE_ERASURES = 1`: job-level erasure bitmap, sampled at `cfg_start` and sliced per beat by the same beat index that builds `keep` – the shape a known-bad column has **TBD / D5**

14.2 AXI4 master ports

Each engine presents one AXI4 master (read-only or write-only) through the house `axi4_master_rd` / `axi4_master_wr` timing wrapper: `AXI_ID_WIDTH`, `AXI_ADDR_WIDTH`, `AXI_DATA_WIDTH` (= `BITS_PER_BEAT` rounded up to a byte multiple), incrementing bursts up to `cfg_xfer_beats`, no narrow transfers. Responses other than OKAY are counted and reported in the job's done status; the data are still passed.

14.3 Parameters

Table 4.7: AXI4 job adapter parameters

Parameter	Default	Meaning
`AXI_ADDR_WIDTH`	32	address width
`AXI_ID_WIDTH`	4	ID width; the engines use one ID
`AXI_MAX_BURST_BEATS`	16	burst cap (`cfg_xfer_beats` may lower it at run time)
`JOB_IF`	`"REGS"`	`"REGS"` or `"DESC"`
`AXI_MONITOR`	0	1 attaches the `_monlite` wrapper variant

14.4 Behaviour

The read engine is modelled on STREAM's `axi_read_engine` and the write engine on its `axi_write_engine` – job valid/address/count in, done strobe out, a burst cap – reduced from STREAM's multi-channel, SRAM-coupled form to one job and a FIFO. Whether that is a reduction of STREAM's modules or a fresh instantiation is decided when the adapter is built. The job controller hands the write engine each block's destination before the block's first bit reaches it, so a decoder job's output blocks land contiguously with the parity removed.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 Register Block

The BCH component has no standalone tops and therefore no register block at revision 0.1. The bare cores are parameterised; their status is the per-block `out_status` port. When the AXI4 job adapter is built a register block will be added to the standalone top, attached through the converters' APB-to-cpuif path exactly as the reed-solomon standalone tops do.

The target register map is the reed-solomon map adapted to bit-level status:

Table 4.8: Register block outline (target)

Register	Access	Contents
`ID`	RO	component ID, version, `KES_ALGO`, `ENABLE_*` build flags
`PROFILE`	RO	m, t, n, k, first root b, primitive polynomial index of the build
`CTRL`	RW	enable, soft reset, erasure enable (if built)
`JOB_SRC`, `JOB_DST`, `JOB_LEN`	RW	job fields (AXI4 ends only)
`JOB_KICK`	WO	start the job
`JOB_STATUS`	RO	busy, done, last job's status summary, AXI response errors
`STAT_BLOCKS`	RO	blocks processed
`STAT_CORRECTED_BLOCKS`	RO	blocks with at least one correction
`STAT_CORRECTED_BITS`	RO	total bits corrected
`STAT_UNCORRECTABLE`	RO	blocks flagged uncorrectable
`STAT_FRAME_ERR`	RO	blocks with a length mismatch
`IRQ_EN`, `IRQ_STATUS`	RW / W1C	interrupt on uncorrectable, on job done, on frame error

Registers will be accessed by name through the generated regmap in DV and host code, never by offset (`vault/handbook/dv/registers-by-name.md`). Counters will be 32-bit, saturating, cleared by `CTRL.soft_reset`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 Clock and Reset

16.1 Single clock

The core and its adapters run on one clock, `aclk`. A consumer whose data path runs on another clock crosses at the consumer's boundary with the repository's CDC blocks (`rtl/cdc`); the core does not contain a crossing. For a standalone top whose AXI4 memory side runs on a different clock than the stream side, the adapter's timing wrapper is the place to insert the `axi4_cdc_{rd,wr}` pair, as the bridge does for its CDC slave ports; that is a build option for the adapter, not a property of the core.

16.2 Reset

`aresetn` is active-low and synchronous, per the component target convention. It is applied through the `ALWAYS_FF_RST` macros with `RST_ASSERTED` so the polarity follows the repository build (`vault/handbook/design/reset-and-clocking.md`). Reset clears control state, the parity and syndrome registers, position and iteration counters and the status. The block buffer's storage is not reset; its pointers are, so stale contents are never read.

16.3 Clock gating

None inside the core. The `_cg` variants of the AXIS and AXI4 wrappers may be selected at the adapters for a standalone top that wants boundary clock gating, following the rest of the repository.

16.4 Frequency

The decoder's clock is set by the key-equation solver's per-iteration critical path, then by the syndrome and Chien cells (one constant multiply plus an XOR). The exact paths are not yet characterised; chapter 5.3 gives the characterisation plan that turns this into a number per part.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

17 Throughput

All throughput numbers are **TBD**. The PRD requires (R5) that no number be claimed before it is measured in an out-of-context synthesis fixture per the repository's practice.

17.1 Sustained rate candidates (PRD D6)

The syndrome pass is the throughput cost: n bit positions are accumulated against t GF MACs. Three architectures are candidates.

Table 5.1: Throughput architecture candidates

Architecture	Bits per cycle	Cost	Best fit
Bit-serial syndrome pass	1	t parallel GF MACs, one per odd syndrome; Chien walks one position per cycle	smallest area; low clock-rate targets
Page-parallel unrolled syndrome tree	n / latency	unrolled GF reduction tree over the whole block; large combinational or deeply pipelined	highest throughput; large area
Symbol-parallel beats with per-beat serial bits	B per beat	B-bit input slice, each bit serially updates the t MACs; Chien evaluates B positions per cycle	balance; matches a byte/word memory datapath

The rate contract – the relationship between the consumer's clock, the block size n, the bits per beat B, and the sustained blocks per second – must be written down once D6 decides. Until then the only honest statement is that the core consumes one beat per cycle when not back-pressured, and the number of bits per beat is itself open (PRD D9).

17.2 Measured placeholder

No synthesis or simulation fixture exists. The table below will be replaced by measured cycles per block after the first RTL and the out-of-context fixture run.

Table 5.1a: Measured cycles per block (placeholder)

Profile	Architecture	Bits/cycle	Cycles per block
CCSDS (63,56)	TBD	TBD	TBD
NAND flash shortened	TBD	TBD	TBD
DVB-T2/S2/C2	TBD	TBD	TBD

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

18 Latency

All latency numbers are **TBD**. The exact values depend on PRD D6 (throughput architecture), D9 (bits per beat), D11 (solver algorithm), and the pipeline depth chosen during micro-architecture.

18.1 Encoder

Table 5.3: Encoder latency (placeholder)

From	To	Cycles
first data bit in	first data bit out	pipeline only
k-th data bit in	first parity bit out	pipeline + 1
k-th data bit in	last parity bit out	pipeline + (n - k) / B

18.2 Decoder

Table 5.4: Decoder latency (placeholder)

Phase	Cycles	Notes
receive the block	n / B	t odd syndromes complete with the last bit
key equation	<= 2t (BM) or data-dependent (Euclid / small-t)	skipped when the syndromes are all zero
Chien walk + bit flip + re-check	n / B	one or more positions per cycle
drain	k / B	the data bits leave while the next block is walked
pipeline	TBD	descriptor skids, solver handoff, verdict stage, output FIFO
first bit in to first bit out	TBD	dominated by 2n/B plus solver and pipeline
first bit in to last bit out	TBD	dominated by 2n/B + k/B plus solver and pipeline

For a shortened code the n above is the shortened length.

18.3 Buffer sizing

The block FIFO must hold the current block while the next arrives during the Chien walk. Its depth is TBD and will be sized once D6 and D9 are fixed. The output FIFO holds a block's data bits while it waits for its verdict and the next block's while that one is walked. The release-on-verdict rule costs this second buffer and extra first-bit latency; it is the price of never emitting an altered bit on an uncorrectable block.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 Resources

All resource numbers are **TBD**. The PRD requires (R5) that no number be claimed before it is measured in an out-of-context synthesis fixture per the repository's practice. Chapter 6.5 is the plan that replaces this table with measurements.

19.1 GF primitives per core

The BCH core reuses the reed-solomon GF primitives (`gf_mul_const`, `gf_mul`, `gf_inv`) without duplicating them. The counts below are the expected structural forms once the RTL exists; they are not measured.

Table 5.5: GF primitive counts (analytic placeholder)

Primitive	Encoder	Decoder	Formula (target)
`gf_mul_const`	2t	2t + t	generator taps + syndrome constant multiplies
`gf_mul`	0	solver-dependent	BM: 6t + 2; Euclid: 8t + 8; small-t: O(t^2) or fewer
`gf_inv`	0	0 or 1	binary BCH needs no Forney inverse; a small-t solver may use one
GF symbol registers	2t	t + solver array	encoder 2t parity; decoder t odd syndromes + locator

19.2 Storage

Table 5.6: Storage (placeholder)

Item	Size	Formula
block FIFO	TBD	(n + latency, rounded up) x B bits, plus erasure bit if enabled
output FIFO	TBD	(2k + latency, rounded up) x (B + 1): corrected bit, hit, last
re-check syndromes	t cells	a second `syndrome_unit` over the corrected stream
log / antilog tables (`gf_inv`)	2 x 2^m x m	LUT ROM at small m, BRAM or Itoh-Tsujii at large m
skid buffers	2 (encoder), 3 (decoder), 2 deep	stage boundaries

19.3 Order of magnitude

A `gf_mul_const` at m = 8 is at most 8 XOR trees of up to 8 inputs; a `gf_mul` is an m x m AND array with reduction. The total arithmetic cost scales with t and m, and with the D6 throughput choice. The first out-of-context run will say where the BCH decoder lands on the target parts.

19.4 Scaling

- m grows: every multiplier grows from m x m, and the log/antilog tables scale as 2^m x m.
- t grows: syndrome cells, solver array and Chien lanes all grow linearly with t.
- Shortening: n is smaller than 2^m - 1, so the Chien walk and buffers are shorter but the arithmetic width is unchanged.
- Bits per beat B (D9): syndrome and Chien datapaths scale with B; the solver is unaffected.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 System Requirements

What a consumer must provide, and what it gets.

20.1 The consumer provides

- **Block boundaries.** `in_last` on the k-th data bit (encoder) or n-th coded bit (decoder) of every block. The consumer's scheduler, framer or job control already knows where blocks end; the core checks and reports, it does not infer.
- **Room for the rate change.** The encoder emits n bits for every k it takes; the decoder emits k for every n. The consumer's downstream (encoder) or upstream (decoder) absorbs the difference, or provides a FIFO of n - k bits so the encoder's parity drain does not stall its input.
- **A single clock** and the repository reset convention (chapter 4.5).
- **Erasure knowledge**, if `ENABLE_ERASURES`: which bits are known bad, per bit or per job, depending on the adapter.
- **Configuration at build time.** The profile is parameters; nothing about the code is set at run time in the bare core.

20.2 The consumer gets

- Systematic encoding: data pass through unchanged and in order, parity follows.
- A per-block verdict on the decoder: ok, corrected (with count), uncorrectable, framing error – valid with the last bit of the block, so the consumer can act on the block it has just received.
- Data as received on an uncorrectable block, never silently altered.
- Determinism: no run-time state survives a block except the counters; the same input block yields the same output block every time.

20.3 Environment

- SystemVerilog-2012, synthesisable, no assertions in RTL (properties live in `formal/`), lint-clean under Verilator per the repository gate.
- Filelists under `rtl/filelists/`, registered in `bin/filelists.toml`, one per top, from the first module.
- Python golden model dependencies (`galois`, `aff3ct`) are DV-only.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 Parameter Configuration

Every parameter of the two cores and the standalone tops. **TBD** marks a PRD decision not yet taken; the value shown is a placeholder, not a default.

21.1 Code

Table 6.1: Code parameters

Parameter	Type	Range	Default	Meaning	PRD
`FIELD_DIM`	int	3..16	**TBD**	m; the field is GF(2^m)	D1
`T_BITS`	int	1..(2^m-1)/2	**TBD**	correctable bit errors per block	D2
`N_BITS`	int	2t+1 .. 2^m-1	**TBD**	block length in bits; < 2^m-1 is a shortened code	D3
`PRIM_POLY`	int	irreducible of degree m	**TBD**	primitive polynomial; selects the field representation	D8
`FIRST_ROOT`	int	0 .. 2^m-2	**TBD**	b, first root of g(x); narrow-sense when b = 1	D8
`ENABLE_RECHECK`	bit	0, 1	1	decoder only: second syndrome unit re-checks the corrected stream and gates the verdict	–

Neither core has an ENABLE_ERASURES or KES_ALGO parameter. Erasures (D5) are not implemented. The solver choice (D11) is bch_key_equation_solver's own KES_ALGO, which only accepts "RIBM" and is not exposed through the decoder core.

Derived: K_BITS = N_BITS - DEGREE_G, where DEGREE_G is the degree of the generator polynomial computed by gf_pkg from m, t, PRIM_POLY and FIRST_ROOT; elaboration fails if K_BITS is below 1.

Every code parameter is fixed at elaboration (PRD D1-D3). There is no run-time t or n: the arrays are generate loops sized by t, the generator polynomial is computed by gf_pkg when the design elaborates, and a block format change is a rebuild.

21.2 Interface / build shape

Table 6.2: Interface parameters

Parameter	Type	Range	Default	Meaning	PRD
`BITS_PER_BEAT`	int	1 .. k	**TBD**	number of bits per valid/ready beat; the bus is a slice of one codeword	D9
`BUILD_ENCODER`	bit		1 **TBD**	include the encoder core	D4
`BUILD_DECODER`	bit		1 **TBD**	include the decoder core	D4
`INTAKE_IF`	string	`"NONE"`, `"AXIS"`, `"AXI4"`	`"NONE"`	intake adapter (standalone tops)	D9
`OUTLET_IF`	string	`"NONE"`, `"AXIS"`, `"AXI4"`	`"NONE"`	outlet adapter	D9

Adapter-specific parameters are in tables 4.5 and 4.7.

21.3 Open decisions in one place

Table 6.3: Open PRD decisions and their placeholders

Decision	What fixes it	Effect if left at the placeholder
D1 field dimension m	the first consumer's page or frame size	GF cell width, table size, Chien length are unresolved
D2 correctable bits t	the consumer's error model	parity count, solver size, Chien width are unresolved
D3 shortening	the consumer's block length	zero-fill on encode, position offset on decode are unresolved
D4 encoder/decoder mix	transmit-only, receive-only, or both	both cores are built; a consumer instantiates what it needs
D5 erasures	storage / RAID consumer says yes	no erasure port, smaller decoder
D6 throughput architecture	the consumer's clock and rate	the whole microarchitecture is unresolved
D8 generator conventions	the standard	field representation and generator taps are unresolved
D9 bits per beat and partial-beat mask	the consumer's datapath width	core port widths and adapter mapping are unresolved
D11 key-equation solver	area / latency target	solver cell counts and cycles are unresolved
D10 first consumer	you	none of the above changes until it appears

22 Candidate Profiles

A profile is a resolved parameter set. These three come from the standards and papers in `References/`; the first consumer picks one or defines its own.

Table 6.4: Candidate profiles

Profile	m	t	n / k	PRIM_POLY	FIRST_ROOT	Source	Notes
CCSDS telecommand	6	1	63 / 56	see figure 3-2 of CCSDS 231.0-B-4 section 3	112 in the CCSDS field representation	CCSDS 231.0-B-4 section 3 (stored in `References/`)	the classic command link: small t, but a fully worked generator in a free standard – the natural first RTL profile
NAND flash ECC	13-16	12-72 class	page-plus-spare geometry, shortened by construction	per controller vendor	per controller vendor	Cai et al. 2017 (stored) for the error landscape; Nabipour & Javidan 2023 (stored) for (m,t)	the profile the memory-controller consumer is expected to name; n runs to

Profile	m	t	n / k	PRIM_POLY	FIRST_ROOT	Source	Notes
						selection	thousands of bits, so D6 and the Chien length dominate
DVB-T2 / S2 / C2	m per standard	per standard	per standard	per standard	per standard	ETSI EN 302 755 / 302 307 / 302 769 (linked in References/)	BCH cleans the LDPC error floor; interesting only if a broadcast consumer ever appears

The CCSDS row is the natural first RTL profile because the generator is explicit in the stored standard. The NAND flash row is the expected consumer but its exact (m, t, n) come from the controller vendor. The DVB row is linked, not stored, because ETSI restricts redistribution.

23 Verification Strategy

23.1 Golden model

Every configuration is verified against a software binary BCH implementation on the same blocks: `galois` (MIT) is the first candidate because it exposes `galois.BCH` with explicit primitive-polynomial, generator and shortening control. `aff3ct` (MIT) is the cross-check. This is the model-first order from the reed-solomon TASK-002 lesson: the golden model is validated before any RTL is compared against it.

The testbench encodes with the model and checks the RTL encoder bit for bit; it injects errors into model-encoded blocks and checks the RTL decoder's corrected output and verdict against the model's decode.

23.2 Test matrix

Table 6.5: Verification matrix

Axis	Values	Why
profile	each row of table 6.4 the build admits	field constants and shortening differ per profile
errors per block	0, 1, t-1, t, t+1, random	0 exercises the bypass; t+1 and more must match the reference decoder's verdict – uncorrectable, or the identical miscorrection where the code itself cannot tell – and an uncorrectable block's data must be exactly as received
reference model	`galois.BCH`	the hardware algorithms in Python on the same field, validated against AFF3CT
erasures (if built)	0 .. 2t, mixed with errors up to 2e + f = 2t	the erasure budget
bits per beat	1, and the consumer's B	the B-fold datapaths
solver	BM, Euclid, small-t where applicable	identical roots on every corrected block
adapters	NONE/NONE first; then AXIS/AXIS and AXI4/AXI4; the mixed pairs follow	the core is proven before any wrapper
back-pressure	none, random on both ends, sustained on the output	the valid/ready contract and the block buffer's full behaviour
framing	short block, long block, missing `last`	the framing-error path

23.3 Test structure

cocotb tests under `dv/tests/` with TB classes in `dv/tbclasses/`, Pattern B (`cocotb_test_*` prefixes), the three run levels through both the REG_LEVEL grid and TEST_LEVEL depth, per the `test-patterns` skill. Traffic at the adapters uses the existing AXI-Stream and AXI4 BFMs; nothing drives a valid/ready port by hand. Registers are read by name through the generated regmap.

23.4 Formal

No assertions in RTL. Candidate properties for `formal/` blocks: the valid/ready contract at both ends of each core; the encoder's parity equals the golden LFSR for one block (bounded); the evenness shortcut S_2j = S_j^2 on the syndrome unit; the equivalence of the solver candidates on symbolic odd syndromes for small t; the no-Forney structural fact that correction is a pure bit flip.

23.5 Coverage

Functional coverage on the matrix above plus: every odd syndrome cell non-zero at least once, every Chien position rooted at least once across the run, the uncorrectable path taken by both the degree check and the root-count check, and the re-check syndrome path when implemented.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

24 Synthesis and Implementation

24.1 Characterisation plan

Numbers in chapter 5 are placeholders. They are replaced by measurements the way the repository measures every other block: an out-of-context synthesis fixture per configuration, on the same two parts the monitor and bridge characterisations use.

Table 6.6: Characterisation steps

Step	Fixture	Output
1	`bch_encoder_core` and `bch_decoder_core`, CCSDS (63,56) profile, B = 1	LUTs, flops, BRAM, WNS; the calibration point
2	decoder with each `KES_ALGO` candidate that is built	the solver comparison in table 3.1 measured
3	B = 8 and B = 64	the bits-per-beat scaling measured once D9 is decided
4	NAND flash shortened profile	large-m, large-t scaling
5	standalone tops with each adapter pair	adapter cost; the monitors' cost is already known

The fixture follows `projects/components/fabric-gen-ip/bridge/fpga/` (`make synth BRIDGE=<fixture>` on `make/fpga_flow.mk`) and reports through the same `summary.csv` shape, so the numbers land beside the bridge's and the monitor's.

24.2 Implementation notes

- All arithmetic is combinational XOR/AND networks; there are no carries, so the multiplier trees of `rtl/math` do not apply and the synthesiser's DSP inference is irrelevant.
- `gf_inv` at small m is two 2^m-entry ROMs and infers to LUTs; at large m it becomes BRAM or Itoh-Tsujii.
- The block buffer infers BRAM once the depth passes the LUT-RAM threshold; `REGISTERED = 1` on the FIFO adds a cycle and removes the read mux from the corrector's path if timing needs it.
- The riBM array has no cross-array feedback, so its cells may be pipelined per iteration if the clock demands; the Euclid array's degree compare cannot be pipelined without the degree-computationless variant.
- No clock gating in the core; boundary gating comes from the `_cg` wrapper variants at the adapters.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

25 Fields and Polynomials

25.1 Why error correction needs a finite field

Every BCH decoder does arithmetic on error patterns. Before we write any hardware, we need a number system where those patterns behave predictably: a field.

A field is a set with two operations (call them addition and multiplication) that satisfy the usual rules — closure, associativity, commutativity, distributivity — plus two extra properties that make algebra work:

- Every element has an additive inverse, so subtraction is always possible.
- Every nonzero element has a multiplicative inverse, so division is always possible.

The integers modulo a prime form a field. The integers modulo a composite do not. Take $mod\,6$: the element 2 has no multiplicative inverse because $gcd(2,6)=2$, and worse, $2\cdot 3\equiv 0\,(mod\,6)$ even though neither factor is zero. Those zero divisors break long division, and long division is exactly what polynomial encoding relies on. In a field, dividing one polynomial by another always produces a unique quotient and remainder; that remainder is what the encoder computes as parity. Without a field, the same division could have multiple remainders, and the decoder would not know which error pattern to blame. Error correction needs a finite field because the block length, the syndrome values, and the hardware datapath all have fixed widths.

25.2 GF(2): the two-element field

Binary BCH codes live in extension fields of $GF(2)$, the field with two elements, 0 and 1. Addition is XOR; multiplication is AND. Both are already hardware primitives.

Table 7.1: GF(2) addition and multiplication

+	**0**	**1**	*	**0**	**1**
0	0	1	0	0	0
1	1	0	1	0	1

Because $1+1=0$, subtraction is the same as addition. This single fact drives almost every simplification in a binary BCH decoder: there are no sign changes, no borrows, and no two's-complement corner cases.

25.3 Polynomials over GF(2)

A polynomial over $GF(2)$ looks like any ordinary polynomial, except its coefficients are only 0 or 1 and all arithmetic on the coefficients is mod 2.

$$f(x) = f_0 + f_1 x + f_2 x^2 + \ldots + f_d x^d, f_i \in \{0, 1\}$$

The degree is the highest power with a nonzero coefficient. A polynomial is irreducible over $GF(2)$ if it cannot be factored into lower-degree polynomials with coefficients in $GF(2)$. Irreducible polynomials play the same role for polynomials that primes play for integers: they are the atoms from which larger structures are built.

For example, $x^2 + x + 1$ is irreducible over $GF(2)$. There is no pair of degree-1 polynomials over $GF(2)$ whose product gives it, because the only degree-1 polynomials are x, $x + 1$, and their products are x^2 and $x^2 + x$ and $x^2 + 1$, none of which equal $x^2 + x + 1$.

Watch out for polynomials that look irreducible but are not. Over $GF(2)$,

$$(x + 1)^2 = x^2 + 2x + 1 = x^2 + 1$$

so $x^2 + 1$ is reducible even though it has no linear factor in the ordinary integer sense. The freshman's dream from section 7.1 makes perfect squares easy to spot: any polynomial whose exponents are all even is the square of another polynomial.

25.4 Building GF(2^m)

We construct $GF(2^m)$ by taking all polynomials of degree less than m over $GF(2)$ and doing arithmetic modulo an irreducible polynomial $p(x)$ of degree m. That makes $GF(2^m)$ a finite field with 2^m elements.

Pick a primitive polynomial $p(x)$ of degree m. The residue of x modulo $p(x)$ is called α:

$$\alpha = x \bmod p(x)$$

Then every nonzero element of $GF(2^m)$ can be written as a power of α, and those powers cycle with period $2^m - 1$:

$$GF(2^m)^{\square} = \{1, \alpha, \alpha^2, \ldots, \alpha^{2^m - 2}\}$$

The element 0 is the extra element that makes the count 2^m. Multiplication is easy in the power representation: $\alpha^i \cdot \alpha^j = \alpha^{i+j \bmod (2^m - 1)}$. Addition is easy in the vector representation: XOR the corresponding bit vectors.

A practical implementation keeps both a log table and an antilog table so it can switch between the two in a cycle or two. To see why both forms matter, multiply $(1+\alpha)$ by $(\alpha+\alpha^2)$. In vector form these are 0011 and 0110 (high-order bit first).

$$(1+\alpha)(\alpha+\alpha^2) = \alpha + \alpha^2 + \alpha^2 + \alpha^3 = \alpha + \alpha^3$$

The two α^2 terms cancel because $1+1=0$. The result is vector 1010, which Table 7.2 identifies as α^9. In the power form the same product is $\alpha^4 \cdot \alpha^5 = \alpha^9$. Both give the same answer; the vector form is how the hardware is wired, and the power form is how the human keeps track.

25.5 Inverses and the log table

Every nonzero element has a multiplicative inverse. In power form this is immediate:

$$\left(\alpha^{i}\right)^{-1} = \alpha^{-i \bmod \left(2^{m}-1\right)} = \alpha^{2^{m}-1-i}$$

So the inverse of α^{5} is α^{10}, and a quick check in Table 7.2 confirms $\alpha^{5} \cdot \alpha^{10} = \alpha^{15} = 1$.

Division is therefore multiplication by the inverse. Solving $\alpha^{7} \cdot z = \alpha^{3}$ for z gives $z = \alpha^{3} / \alpha^{7} = \alpha^{3-7 \bmod 15} = \alpha^{11}$. The decoder uses this constantly: the Peterson solve in section 7.3 divides by a determinant, and every such division is one inverse lookup followed by one multiplication.

If you prefer not to store a log table, the inverse can also be computed with the extended Euclidean algorithm on polynomials over $GF(2)$. For a small fixed field the table is smaller and faster, so that is what the RTL does.

25.6 The freshman's dream in characteristic 2

In a field of characteristic 2, the binomial theorem collapses because all intermediate binomial coefficients are even:

$$(a+b)^2 = a^2 + 2ab + b^2 = a^2 + b^2$$

More generally, for any non-negative integer j:

$$(a+b)^{2^j} = a^{2^j} + b^{2^j}$$

This identity is not a trick; it is the reason binary BCH decoders only need to compute half the syndromes. If S_j is one syndrome, then $S_{2j} = S_j^2$ is free. The decoder hardware computes the odd-indexed syndromes and derives the even ones by squaring. We revisit that shortcut in section 7.3.

25.7 A concrete field: GF(2^4)

For the worked examples in this chapter we use $m = 4$ and the primitive polynomial

$$p(x) = x^4 + x + 1$$

which is 10011 in binary. The element $\alpha = x \bmod p(x)$ is represented by the 4-bit vector 0010. Repeated multiplication by α generates all 15 nonzero elements.

Table 7.2: Log/antilog table for GF(2^4) with p(x) = x^4 + x + 1

i	alpha^i as polynomial	4-bit vector
0	1	0001
1	alpha	0010
2	alpha^2	0100
3	alpha^3	1000
4	1 + alpha	0011
5	alpha + alpha^2	0110
6	alpha^2 + alpha^3	1100
7	1 + alpha + alpha^3	1011
8	1 + alpha^2	0101
9	alpha + alpha^3	1010
10	1 + alpha + alpha^2	0111
11	alpha + alpha^2 + alpha^3	1110
12	1 + alpha + alpha^2 + alpha^3	1111
13	1 + alpha^2 + alpha^3	1101
14	1 + alpha^3	1001

Two checks against the table. First, α^4 must satisfy $p(\alpha) = 0$, so $\alpha^4 = \alpha + 1$, which is vector 0011 — exactly the row for $i = 4$. Second, addition is XOR: α^3 is 1000 and α^5 is 0110, so their sum is 1110, which the table identifies as α^{11}.

Table 7.3: Addition examples in GF(2^4)

a	b	a + b (XOR)	result
alpha^3	alpha^5	1000 xor 0110	alpha^11

a	b	a + b (XOR)	result
		= 1110	
alpha^7	alpha^13	1011 xor 1101 = 0110	alpha^5
alpha^4	alpha^8	0011 xor 0101 = 0110	alpha^5

Multiplication is equally mechanical. To multiply α^7 by α^{13} we add exponents mod 15: $7+13=20\equiv 5\left(mod\,15\right)$, so the product is α^5. To multiply two vector forms without the table, multiply the polynomials and reduce modulo $p\left(x\right)$. Hardware rarely does it that way; it uses the log/antilog ROM or a fixed XOR network.

25.8 Why this is hardware-shaped

This chapter is the only place the decoder leaves the binary world and operates on m-bit field elements, but the hardware mapping is direct:

- An m-bit register holds one field element.
- Addition is m XOR gates. There are no carries anywhere in the codec; this is field addition, not binary arithmetic.
- Multiplication by a constant is a fixed XOR network. The constant is hard-wired from the profile parameters.
- General multiplication uses an AND array followed by a reduction tree.
- Inversion is usually a table lookup.

The absence of carries is worth emphasizing. A binary adder chains carries from bit 0 to bit $m-1$; a field adder XORs every bit independently. That makes the GF datapath fast and easy to pipeline, which is why the encoder and decoder can afford one symbol operation per cycle even for large m .

Multiplication by the constant α — the operation the LFSR does on every clock edge — is a good illustration. In $GF(2^4)$ with $p(x)=x^4+x+1$, multiplying a vector (b_3, b_2, b_1, b_0) by α shifts it left and, if the bit shifted out is 1, XORs 0011 into the low bits. The whole thing is four 2-input XOR gates and a few muxes. A full variable multiply needs more logic, but it is still pure combinatorial gates; no iteration, no microcode.

The `gf_*` primitives that implement these operations live in the reed-solomon component tree and are imported into the BCH core, as described in chapter 3.1's GF layer subsection. Every arithmetic block in the encoder and decoder is built from three of them: `gf_mul_const` for multiplication by a known constant, `gf_mul` for general multiplication, and `gf_inv` for inversion. The `gf_pkg` package holds the constant functions that evaluate this arithmetic — and the generator polynomial coefficients derived from t, b, and p(x) — from the profile parameters discussed in chapter 6.2.

26 From Field to Code

26.1 Codewords as polynomials

A binary BCH block of n bits is a polynomial over $GF(2)$:

$$c(x) = c_0 + c_1 x + c_2 x^2 + \ldots + c_{n-1} x^{n-1}, c_i \in \{0, 1\}$$

The least significant bit is c_0; the most significant bit is c_{n-1}, which is also the first transmitted bit. This is not a general-purpose polynomial — it is just a convenient way to talk about a bit vector and its shifts at the same time.

These codes are cyclic. Multiply $c(x)$ by x and reduce modulo $(x^n - 1)$:

$$x \cdot c(x) \bmod (x^n - 1)$$

Because we are in characteristic 2, $x^n - 1 = x^n + 1$. The reduction wraps the coefficient of x^n around to the constant term, which is exactly a cyclic rotation of the n bits. If $c(x)$ is a codeword, so is every cyclic shift of it.

For a concrete 15-bit example, suppose $c(x)$ has a 1 at x^{14} and nowhere else. Then $x \cdot c(x) = x^{15}$, and reducing modulo $x^{15} + 1$ gives 1, which places the 1 at the constant term. The bit moved from polynomial exponent 14 to polynomial exponent 0. That cyclic property is why the encoder is a feedback shift register and why the decoder can treat a block as a single polynomial.

Every valid codeword is a multiple of the generator polynomial $g(x)$. If $u(x)$ is any polynomial of degree less than k, then $c(x) = u(x) \cdot g(x)$ is a codeword. There are 2^k such polynomials, so there are 2^k codewords. The generator polynomial therefore completely defines the code; the data bits only select which multiple of $g(x)$ we transmit. Systematic encoding chooses $u(x)$ so that the high coefficients of the product are exactly the data bits, then adds the remainder $r(x)$ to make the low coefficients the parity.

26.2 Generator polynomial and consecutive roots

A BCH code is defined by its generator polynomial $g(x)$, a polynomial over $GF(2)$ whose roots are $2t$ consecutive powers of α:

$$g(\alpha^b) = g(\alpha^{b+1}) = \ldots = g(\alpha^{b+2t-1}) = 0$$

The parameter b is the first root. When $b = 1$ the code is narrow-sense, the default in most textbook treatments and in this chapter's example. Chapter 6.3 notes that CCSDS telecommand uses $b = 112$ in its field representation; the math is the same, only the exponents shift.

The number of consecutive roots determines the guaranteed error-correction capability. A generator with $2t$ consecutive roots produces a code with designed distance $2t+1$, which means any two valid codewords differ in at least $2t+1$ bit positions. The proof is the BCH bound; the intuition is that $2t$ roots give the decoder enough independent equations to solve for t error locations. For our example $t = 2$, so the designed distance is 5: any two valid codewords differ in at least five bit positions. That guarantees correction of up to 2 errors and detection of up to 4 errors. We do not prove the bound here, but we use it to choose t for the target bit-error rate.

26.3 Minimal polynomials

Each root α^i has a minimal polynomial $m_i(x)$: the monic irreducible polynomial over $GF(2)$ that has α^i as a root in $GF(2^m)$. Because the coefficients must lie in $GF(2)$, the other roots of $m_i(x)$ are forced to be the conjugates of α^i under squaring. Here is why. If $m_i(x)$ has coefficients in $GF(2)$ and $m_i(\alpha^i)=0$, then applying the freshman's dream from section 7.1 gives

$$0=\left(m_i(\alpha^i)\right)^2=m_i\left((\alpha^i)^2\right)=m_i(\alpha^{2i})$$

so α^{2i} is also a root. Repeating the argument collects the whole cyclotomic coset.

$$\alpha^i,\alpha^{2i},\alpha^{4i},\alpha^{8i},\ldots$$

taken modulo 2^m-1 until the sequence repeats. That set is the cyclotomic coset of i.

For $i=1$ in $GF(2^4)$ the coset is $\{1,2,4,8\}$, so

$$m_1(x)=(x+\alpha)(x+\alpha^2)(x+\alpha^4)(x+\alpha^8)$$

Multiplying this out and reducing every coefficient modulo 2 gives x^4+x+1. For $i=3$ the coset is $\{3,6,12,9\}$, and the same procedure gives $x^4+x^3+x^2+x+1$.

The generator is the least common multiple of the minimal polynomials of the consecutive roots:

$$g(x)=lcm\{m_b(x),m_{b+1}(x),\ldots,m_{b+2t-1}(x)\}$$

Since distinct minimal polynomials are irreducible, the lcm is just their product. The degree of $g(x)$ is the number of parity bits:

$$deg(g)=n-k\le m\cdot t$$

For the BCH(15, 7) example in this chapter, $m=4$ and $t=2$, so $deg(g)=8=4\cdot 2$.

Table 7.4: Minimal polynomials and cyclotomic cosets for alpha^1 .. alpha^4 in GF(2^4)

i	cyclotomic coset	minimal polynomial m_i(x)
1	{1, 2, 4, 8}	x^4 + x + 1
2	{1, 2, 4, 8}	x^4 + x + 1
3	{3, 6, 12, 9}	x^4 + x^3 + x^2 + x + 1
4	{1, 2, 4, 8}	x^4 + x + 1

The coset for $i=1$ has four members, so $m_1(x)$ has degree 4. The same coset catches α^2 and α^4, which is why their minimal polynomials are identical. The coset for $i=3$ is disjoint, giving a second degree-4 minimal polynomial.

26.4 Constructing BCH(15, 7)

We build the narrow-sense code with $n = 15 = 2^4 - 1$, $k = 7$, and $t = 2$. The consecutive roots are $\alpha^1, \alpha^2, \alpha^3, \alpha^4$. From Table 7.4 only two distinct minimal polynomials are needed:

$$g(x) = lcm\left(m_1(x), m_3(x)\right) = m_1(x) \cdot m_3(x)$$

$$g(x) = \left(x^4 + x + 1\right)\left(x^4 + x^3 + x^2 + x + 1\right)$$

Multiplying the two degree-4 polynomials gives terms from x^8 down to x^0. In characteristic 2, any coefficient that appears twice cancels. The x^5, x^3, x^2, and x^1 terms each appear an even number of times and disappear; the remaining terms are:

$$g(x) = x^8 + x^7 + x^6 + x^4 + 1$$

The coefficients from x^8 down to x^0 are:

Table 7.5: Coefficients of g(x) = x^8 + x^7 + x^6 + x^4 + 1

Power	x^8	x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
Coeff	1	1	1	0	1	0	0	0	1

The degree is 8, exactly $n - k = 15 - 7$, so this code has 8 parity bits and 7 data bits.

A full-length primitive code has $n = 2^m - 1$. Most real profiles are shortened: the configured block length is smaller, and the missing leading bits are treated as zero in the syndrome computation and Chien search. Chapter 6.2 covers the parameters and chapter 6.3 lists the candidate profiles. The algebra does not change; only the position offsets and the assumption of leading zeros change.

26.5 Systematic encoding

A systematic codeword keeps the original k data bits unchanged and appends $n-k$ parity bits. Write the data polynomial as $d(x)$ with degree less than k. Shift it by $n-k$ positions:

$$x^{n-k} \cdot d(x)$$

Divide by $g(x)$ and keep the remainder $r(x)$, which has degree less than $n-k$:

$$r(x) = x^{n-k} \cdot d(x) \bmod g(x)$$

The codeword is then:

$$c(x) = x^{n-k} \cdot d(x) + r(x)$$

The high part carries the data; the low part carries the parity. Because $c(x)$ differs from $x^{n-k}d(x)$ by a multiple of $g(x)$, it is a multiple of $g(x)$ and therefore a valid codeword. Every root of $g(x)$ is also a root of $c(x)$, which is why the syndrome test in section 7.3 works: evaluating $c(x)$ at any root of $g(x)$ gives zero.

The LFSR is polynomial long division in hardware. Each incoming bit shifts the partial dividend left by one power of x. If the bit shifted out of the high position is 1, the register XORs the generator coefficients into the lower positions — that is the same as subtracting $g(x)$ from the current partial remainder. After the last data bit the register contains exactly the remainder $r(x)$.

The hardware in chapter 3.2 is exactly this division. The encoder core's parity registers form a bit-level LFSR whose feedback taps are the binary coefficients of $g(x)$: each arriving bit shifts the partial remainder up one power of x, and when the bit that falls off the end is a 1, the tap positions XOR $g(x)$ back in. Because the code is binary, that feedback is plain XOR — the binary encoder contains no GF multipliers at all, which is the structural difference from the RS encoder, whose symbol-level LFSR multiplies by the field-element coefficients of its $g(x)$.

26.6 Worked example: encoding one block

Take the 7-bit message

$$d(x) = x^6 + x^3 + x^2 + 1$$

In bit order $d_6 d_5 d_4 d_3 d_2 d_1 d_0$ this is 1001101. Feed the bits into the encoder LFSR one at a time, most significant bit first—this is the same as transmission order. The generator is $g(x) = x^8 + x^7 + x^6 + x^4 + 1$, so the feedback taps are at positions 7, 6, 4, and 0. The x^8 term is implicit in the shift register: it is the bit that falls off the left end and becomes the feedback decision.

Table 7.6: LFSR contents while encoding d(x) = x^6 + x^3 + x^2 + 1

Step	Input bit	LFSR p7..p0 after update
1	1	11010001
2	0	01110011
3	0	11100110
4	1	11001100
5	1	10011000
6	0	11100001
7	1	11000010

The LFSR table and a manual polynomial long division give the same remainder, which is a useful cross-check. The final register contents 11000010 are the parity polynomial:

$$r(x) = x^7 + x^6 + x$$

Appending the parity to the data gives the systematic 15-bit codeword:

$$c(x) = x^{14} + x^{11} + x^{10} + x^8 + x^7 + x^6 + x$$

In bit order $c_{14} \ldots c_0$:

$$c = 100110111000010$$

The first seven bits are the original message and the last eight bits are the parity. We can sanity-check the result by evaluating $c(x)$ at a generator root such as α. Every term in $c(x)$ is a power of α; using Table 7.2 from section 7.1 to convert those powers to vectors and XORing them gives 0. That is not an accident — it is the definition of a valid codeword. We use exactly this 15-bit word in section 7.3, where we corrupt two bits and walk through the decode.

27 Syndromes, the Key Equation, and Correction

27.1 Received polynomial and syndromes

The channel flips some bits. The decoder receives a polynomial $R(x)$ that is the transmitted codeword $c(x)$ plus an error polynomial $e(x)$:

$$R(x) = c(x) + e(x)$$

The error polynomial has a 1 at every flipped bit position and 0 everywhere else. If no errors occurred, $e(x) = 0$ and $R(x) = c(x)$.

The specification writes the syndromes as $S_i = R\left(\alpha^{b+i}\right)$ for $i = 0 \ldots 2t - 1$, where b is the first root. For narrow-sense codes $b = 1$, and it is often cleaner to index by the exponent itself. In the rest of this chapter we use

$$S_i = R\left(\alpha^{i}\right), i = 1 \ldots 2t$$

This is the same set of values, just renumbered. Because $c(x)$ is divisible by $g(x)$, every root of $g(x)$ is also a root of $c(x)$, so evaluating $c(x)$ at those roots gives zero. The syndromes therefore depend only on the error polynomial:

$$S_i = R\left(\alpha^{i}\right) = c\left(\alpha^{i}\right) + e\left(\alpha^{i}\right) = e\left(\alpha^{i}\right)$$

All syndromes are zero if and only if $R(x)$ is a valid codeword. That is the first test the decoder performs, and it is the reason the syndrome unit of chapter 3.2 exists.

The unit evaluates $R(x)$ at the t independent roots as the bits stream in. It does not store the whole block and then substitute. Instead it uses Horner's rule:

$$S_i = \left(\ldots \left(\left(r_{n-1} \alpha^{i} + r_{n-2} \right) \alpha^{i} + r_{n-3} \right) \ldots \right) \alpha^{i} + r_0$$

Bits arrive most-significant first, so r_{n-1} is the first transmitted bit. Each arriving bit r_j triggers one multiply by the constant α^{i} and one XOR into the accumulator. After the last bit the accumulator holds S_i. The multiply-by-constant is a fixed XOR network, so the syndrome cell is small and fast.

27.2 The evenness shortcut

In characteristic 2, the freshman's dream from section 7.1 gives:

$$(a+b)^2 = a^2 + b^2$$

Apply this to a syndrome. If $e(x)$ has errors at polynomial exponents $j_1, j_2, \ldots$, then

$$S_i = \alpha^{i j_1} + \alpha^{i j_2} + \ldots$$

Squaring the syndrome doubles every exponent:

$$S_i^2 = \alpha^{2 i j_1} + \alpha^{2 i j_2} + \ldots = S_{2i}$$

So the even-indexed syndromes are not independent. For $t = 2$:

$$S_2 = S_1^2, S_4 = S_2^2 = S_1^4$$

Once S_1 and S_3 are known, S_2 and S_4 come for free by squaring. The syndrome unit in chapter 3.2 therefore has only t cells, not $2t$; each cell accumulates one odd-exponent syndrome. That is the evenness shortcut defined in chapter 1.3.

27.3 The key equation

The error-locator polynomial $\Lambda(x)$ is the monic polynomial whose roots are the error locators $\alpha^{j_1}, \alpha^{j_2}, \ldots$, where each j_l is a polynomial exponent:

$$\Lambda(x) = \left(x + \alpha^{j_1}\right)\left(x + \alpha^{j_2}\right) \cdots$$

It satisfies the key equation:

$$\Lambda(x) \cdot S(x) \equiv \Omega(x) \left(mod\, x^{2t}\right)$$

where $S(x)$ is the syndrome polynomial and $\Omega(x)$ is the error-evaluator polynomial. In words: $\Lambda(x)$ is the smallest polynomial that, when multiplied by the syndrome polynomial, leaves only low-degree terms. Its roots tell us where the errors are.

In a reed-solomon decoder, $\Omega(x)$ is also needed because the error magnitude at each location can be any field element. In a binary BCH decoder the magnitude is always 1, so once the Chien search finds the positions the corrector simply flips the bits. The solver only has to deliver $\Lambda(x)$.

The key equation can be inverted by several algorithms. Chapter 3.3 compares three candidates for this design:

- **riBM** (reformulated inversionless Berlekamp-Massey) is the default general-purpose choice.
- **Euclidean / ME** (modified Euclidean) is a readable alternative that avoids division.
- **PGZ / step-by-step** is attractive for small t because it solves a small linear system directly.

For the worked example below we use the Peterson-Gorenstein-Zierler direct solve. It is the same algorithm the `SMALL_T` solver candidate would use, and every step is visible in the field arithmetic. Berlekamp-Massey iteratively builds $\Lambda(x)$ by inspecting one syndrome discrepancy at a time; the Euclidean algorithm repeatedly cross-multiplies two polynomials until one of them becomes the locator; PGZ writes the syndrome identities as a small linear system and solves it directly with field operations.

27.4 Chien search and binary correction

After the solver produces $\Lambda(x)$, the Chien search evaluates it at every polynomial exponent:

$$\Lambda(\alpha^0), \Lambda(\alpha^1), \ldots, \Lambda(\alpha^{n-1})$$

If $\Lambda(\alpha^j) = 0$, the coefficient of x^j—that is, bit j in polynomial-exponent form—is in error. The Chien search walks the n positions in lock-step with the block buffer read-out, one position per cycle in the serial architecture or several positions per cycle if the throughput profile selects a parallel evaluation tree. Because the code is binary, every error value is 1, so correction is a pure flip: the corrector XORs a 1 into that bit as it leaves the block buffer. There is no Forney stage and no magnitude computation. That is the main mathematical difference between this binary BCH decoder and the reed-solomon decoder, which must also compute how much each symbol was corrupted.

27.5 Worked example: correcting two errors

Position convention. In this example, "bit j" means the coefficient of x^j in the codeword polynomial. The RTL counts transmission-order positions $p = 0, 1, \ldots$ starting with the first transmitted bit, so coefficient x^j corresponds to position

$$p = n - 1 - j$$

For the two flipped bits, exponents $j = \{10, 3\}$ map to RTL positions $p = \{4, 11\}$.

We start with the 15-bit codeword from section 7.2:

$$c = 100110111000010$$

Flip the bits whose polynomial exponents are 10 and 3—that is, add $x^{10} + x^3$ to $c(x)$. The received polynomial is:

$$R(x) = c(x) + x^{10} + x^3$$

In bit order $R_{14} \ldots R_0$ the received word is:

$$R = 100100111001010$$

27.5.1 Computing the odd syndromes

We need $S_1 = R(\alpha)$ and $S_3 = R(\alpha^3)$. Using Table 7.2 from section 7.1 to replace each power of α by its 4-bit vector and XORing the terms that are present in $R(x)$:

Table 7.7: Odd syndromes of the received word

Syndrome	Exponent sum	Vector	Value
S_1	alpha^12	1111	alpha^12
S_3	alpha^7	1011	alpha^7

The value $S_1 = 1111$ comes from XORing the vectors for every set bit exponent in $R(x)$: $\alpha^{14} = 1001$, $\alpha^{11} = 1110$, $\alpha^8 = 0101$, $\alpha^7 = 1011$, $\alpha^6 = 1100$, $\alpha^3 = 1000$, and $\alpha^1 = 0010$. Chasing the accumulating XOR gives $1111 = \alpha^{12}$. For S_3 we use the same exponents but with tripled values mod 15, which is what the syndrome cell with root α^3 accumulates.

The even syndromes follow from the shortcut:

$$S_2 = S_1^2 = (\alpha^{12})^2 = \alpha^9 = 1010$$

$$S_4 = S_2^2 = (\alpha^9)^2 = \alpha^3 = 1000$$

So the syndrome unit only had to compute two 4-bit values, S_1 and S_3; the other two syndromes came from two field squarings.

27.5.2 Solving the t = 2 Peterson system

For two errors the recurrence is:

$$S_{j+2} + \Lambda_1 S_{j+1} + \Lambda_2 S_j = 0,\ j = 1, 2$$

which gives the linear system over $GF(2^4)$:

$$\Lambda_1 S_2 + \Lambda_2 S_1 = S_3$$

$$\Lambda_1 S_3 + \Lambda_2 S_2 = S_4$$

Substitute the syndrome values $S_1 = \alpha^{12}$, $S_2 = \alpha^9$, $S_3 = \alpha^7$, $S_4 = \alpha^3$:

$$\Lambda_1 \alpha^9 + \Lambda_2 \alpha^{12} = \alpha^7$$

$$\Lambda_1 \alpha^7 + \Lambda_2 \alpha^9 = \alpha^3$$

The determinant is:

$$\Delta = S_2^2 + S_1 S_3 = \alpha^3 + \alpha^4 = \alpha^7$$

Using Cramer's rule over the field:

$$\Lambda_1 = \frac{S_2 S_3 + S_1 S_4}{\Delta} = \frac{\alpha^9 \cdot \alpha^7 + \alpha^{12} \cdot \alpha^3}{\alpha^7} = \frac{\alpha + 1}{\alpha^7} = \frac{\alpha^4}{\alpha^7} = \alpha^{12}$$

$$\Lambda_2 = \frac{S_2 S_4 + S_3^2}{\Delta} = \frac{\alpha^9 \cdot \alpha^3 + (\alpha^7)^2}{\alpha^7} = \frac{\alpha^{12} + \alpha^{14}}{\alpha^7} = \frac{\alpha^5}{\alpha^7} = \alpha^{13}$$

So the error-locator polynomial is:

$$\Lambda(x) = x^2 + \alpha^{12} x + \alpha^{13}$$

Any nonzero scalar multiple of $\Lambda(x)$ has the same roots, so the Chien search and the corrector do not care about the exact scale. The riBM hardware in section 7.5 produces a scaled reciprocal of this form; the verdict logic only counts roots and checks degree, both of which are scale-invariant.

27.5.3 Chien search

Now evaluate $\Lambda(x)$ at α^j for $j = 0 \ldots 14$.

Table 7.8: Chien search over the 15 bit positions

j	alpha^j vector	Lambda(alpha^j)	Note
0	0001	0011	
1	0010	0100	
2	0100	0111	
3	1000	0000	ROOT
4	0011	1010	
5	0110	1110	
6	1100	1010	
7	1011	0111	
8	0101	1001	
9	1010	1001	
10	0111	0000	ROOT
11	1110	0011	
12	1111	1101	
13	1101	0100	
14	1001	1110	

The zeros occur at $j=3$ and $j=10$, exactly the exponents we flipped (RTL positions $p=11$ and $p=4$). No other position yields zero, so the decoder does not invent extra errors. The corrector XORs those two bits, restoring:

$$1001101\,11000010$$

A final syndrome check over the corrected block gives all four syndromes $S_1=S_2=S_3=S_4=0$, confirming the result is a valid 15-bit codeword. The corrected bit vector matches the original codeword from section 7.2 exactly.

27.6 When correction fails

Not every received block can be corrected. The decoder declares a block uncorrectable when the math does not line up. Two common cases:

- The locator polynomial $\Lambda(x)$ has degree v but fewer than v distinct roots in the field. Then the error count implied by the degree does not match the actual root count.
- The locator has degree greater than t. Even if it has roots, the code was only designed to correct t errors, so those roots are not trustworthy.

Chapter 3.2 describes how the verdict logic flags these cases. Some blocks with more than t errors happen to produce a locator with the right number of roots but the wrong positions; the decoder catches those by running a second syndrome check over the corrected stream. An uncorrectable block leaves the buffer unchanged; the status port reports the failure with the final bit.

27.7 From math to hardware

Table 7.9: Mapping the decode math to the decoder blocks

Math step	Hardware block	Where it lives
Syndrome evaluation S_i = R(alpha^i)	syndrome unit	chapter 3.2
Key-equation solve for Lambda(x)	key-equation solver	chapter 3.3
Root search Lambda(alpha^j) = 0	Chien search + corrector	chapter 3.2
Block storage and playback	block buffer	chapter 3.2
Final verdict and status	status ports	chapter 3.2

The syndrome unit and Chien search are pure field arithmetic pipelines. The solver is the only block whose internal algorithm varies across the candidate profiles; everything downstream sees only the locator polynomial and the syndromes. That separation is what lets the architecture in chapter 3.1 swap riBM, Euclidean, or PGZ without touching the buffer, the corrector, or the interface in chapter 4.1.

28 The Parameters n, k, t, m

Every BCH code boils down to a handful of integers. Here's what each one means, how they fit together, and where they show up in the RTL.

28.1 n: block length

n is the number of bits in one coded block. For a primitive BCH code it reaches the full length of the field:

$$n_{\text{full}} = 2^m - 1$$

Most real profiles are shortened: the configured n is smaller than $2^m - 1$, and the missing high-order bits are treated as zero during syndrome computation and Chien search. The algebra doesn't change—only the position offsets and the assumption of leading zeros do. The package computes the full-field size in `bch_order` and the configured size is the parameter `N_BITS` (`rtl/fub/bch_pkg.sv:50-52`, `rtl/fub/bch_pkg.sv:37`).

28.2 m: field dimension

The code lives in $GF(2^m)$. Each nonzero field element is a power of a primitive element α, and every element is stored as an m-bit vector. The primitive polynomial $p(x)$ of degree m is encoded as `PRIM_POLY`; bit m is the leading term and is implicit. The default profile uses $m=6$ and $p(x)=x^6+x+1$ (`PRIM_POLY = 0x43`) (`rtl/fub/bch_pkg.sv:34-35`).

28.3 k: message length

k is the number of data bits carried inside one n-bit block. It isn't an independent parameter; it's fixed by the degree of the generator polynomial:

$$k = n - deg(g)$$

The encoder core exposes `K_BITS` as exactly this difference (`rtl/macro/bch_encoder_core.sv:71`, `rtl/macro/bch_decoder_core.sv:87`).

28.4 t: correctable bit errors

t is the guaranteed number of bit flips the decoder can fix. The generator has $2t$ consecutive roots:

$$g(\alpha^b) = g(\alpha^{b+1}) = \ldots = g(\alpha^{b+2t-1}) = 0$$

The first root b is `FIRST_ROOT`. When $b = 1$ the code is narrow-sense, the usual textbook case. When $b = 0$ the consecutive range includes $\alpha^0 = 1$, so $g(x)$ gains an $(x+1)$ factor and can detect an extra odd-weight error pattern. The designed minimum distance is

$$d \geq 2t + 1$$

This is the BCH bound; the design chooses the roots and therefore t (`rtl/fub/bch_pkg.sv:56-59`).

28.5 deg g: parity bits

The degree of the generator is the number of parity bits appended to each block:

$$deg(g) = n - k$$

The degree counts the distinct conjugates of the consecutive roots $\alpha^{b} \dots \alpha^{b+2t-1}$. Each cyclotomic coset modulo $2^{m} - 1$ contributes its size once, so the degree is bounded by

$$deg(g) \le m \cdot t$$

For many profiles the bound is loose; for the Genesys 2 profile below it's tight: $deg(g) = 13 \cdot 8 = 104$. The package computes the degree without expanding the whole polynomial when possible (`rtl/fub/bch_pkg.sv:62-83`).

28.6 Parameters in the RTL

Table 7.10: RTL parameter names for the BCH code variables

Symbol	RTL parameter	Meaning
m	`FIELD_DIM`	Dimension of $GF(2^m)$
$p(x)$	`PRIM_POLY`	Primitive polynomial of degree m
t	`T_BITS`	Correctable bit errors
n	`N_BITS`	Configured codeword length
b	`FIRST_ROOT`	First consecutive root of $g(x)$
k	`K_BITS`	Derived as $n - deg(g)$
B	`BITS_PER_BEAT`	Bits per AXI-Stream/AXI4 beat

`BITS_PER_BEAT` is bits per beat, not symbols. The AXI-Stream wrapper defaults it to 8 (`rtl/top/bch_encoder_axis4.sv:48-69`) and the AXI4 wrapper defaults the bus width to 8 (`rtl/top/bch_encoder_axi4.sv:39-56`).

28.7 Table of built profiles

Table 7.11: Built BCH profiles

Profile	m	PRIM_POLY	t	b	n	k	$deg(g)$	g (hex)
CCSDS TC narrow-sense	6	0x43	1	1	63	57	6	0x43
CCSDS TC modified	6	0x43	1	0	63	56	7	0xC5
Mid-size sanity	6	0x43	2	1	63	51	12	0x1539
Genesys 2 board	13	0x201B	8	1	4224	4120	104	(see package)

The Genesys 2 board loopback design runs the shortened profile $\mathrm{BCH}(4224, 4120)$ with $t = 8$ (`projects/fpga-systems/Genesys2/ecc-ip/bch/build-loop/rtl/bch_loop_cfg_pkg.sv:26-31`). The $m = 13$ primitive polynomial is $p(x) = x^{13} + x^4 + x^3 + x + 1$ (`0x201B`). The degree lands exactly at $m \cdot t = 104$, so the parity budget is fully used.

28.8 Choosing the parameters: a worked example

Say a 512-byte sector has to survive up to 8 bit flips. The data payload is $512 \cdot 8 = 4096$ bits.

1. Pick the field. $m = 12$ gives $n_{\text{full}} = 4095$, which isn't enough room for both data and parity. $m = 13$ gives $n_{\text{full}} = 8191$, comfortably larger than 4096.
2. Pick t. With $t = 8$ the BCH bound promises correction of up to 8 bit errors. The parity budget is at most $m \cdot t = 104$ bits.
3. Choose the block length. A 4096-bit payload plus a 128-bit spare region gives $n = 4224$.
4. Check the degree. For $m = 13$, $t = 8$, $b = 1$, the generator degree is exactly 104, so $k = 4224 - 104 = 4120$. The 4096 data bits plus 24 unused bits in the data region are carried as the k message bits.

That's the Genesys 2 profile in Table 7.11.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

29 Every Stage: Math to Pseudocode

Here's the bridge between the math in sections 7.1–7.3 and the Verilog in chapter 3.2. For each block you'll see the equation it implements, pseudocode that uses the same register names as the RTL, and a one-line hardware mapping.

29.1 Encoder (bch_encoder_core)

The encoder keeps the first k input bits untouched and appends the parity remainder. Bits arrive most-significant first—the same order they leave the chip.

The math. Shift the message polynomial $d(x)$ by $n-k$ positions and divide by $g(x)$:

$$r(x) = x^{n-k} \cdot d(x) \bmod g(x)$$

$$c(x) = x^{n-k} \cdot d(x) + r(x)$$

The LFSR is polynomial long division in hardware.

The pseudocode. `r_lfsr[j]` is lane j of the remainder register; lane 0 is the low-order coefficient and lane `DEG_G-1` is the coefficient that shifts out first during the parity drain. `GEN_BITS[j]` is the binary tap g_j.

```
function lfsr_step(r_lfsr, d):
    fb = d ^ r_lfsr[DEG_G - 1]
    for j = 0 .. DEG_G - 1:
        prev = (j == 0) ? 0 : r_lfsr[j - 1]
        r_lfsr[j] = GEN_BITS[j] ? prev ^ fb : prev
    return r_lfsr

// beat unroll: B bits per cycle
w_chain[0] = r_lfsr
for u = 0 .. B - 1:
    w_chain[u + 1] = lfsr_step(w_chain[u], in_data[u])
w_lfsr_next = w_chain[B]
for u = 1 .. B:
    if w_in_count == u:
        w_lfsr_next = w_chain[u]

r_lfsr <= w_lfsr_next           // on every accepted data beat

// parity drain
w_par_beat_len = min(B, remaining parity bits)
for u = 0 .. w_par_beat_len - 1:
    w_parity_bits[u] = r_lfsr[DEG_G - 1 - u]
r_lfsr <= r_lfsr << w_par_beat_len
```

In the RTL. `lfsr_step` lives at `rtl/macro/bch_encoder_core.sv:143-160`; the unrolled chain and partial-beat mux are at `:175-186`; the parity drain reads the high-order lanes at `:231-238`; and `frame_err` fires on a bit-count mismatch at `:228`. The hardware is a single GF(2^m) bit-LFSR unrolled `BITS_PER_BEAT` times. Here's the part that bites: `w_chain` is purely combinational—only `r_lfsr` and the counters are clocked, so don't try to pipeline the chain itself.

29.2 Syndrome unit (bch_syndrome_unit)

The syndrome unit evaluates the received polynomial $R(x)$ at the t independent odd roots. The even syndromes aren't computed here; they're rebuilt later by squaring.

The math. For each odd root exponent j in the consecutive range:

$$S_j = R(\alpha^j) = \sum_{p=0}^{n-1} R_p \cdot \alpha^{j \cdot p}$$

The even syndromes are not computed here; they are rebuilt later by squaring.

The pseudocode. One `gf_syndrome_cell` per odd root. `ROOT` is the fixed field element $\alpha^{\text{root_exp}}$.

```
for each odd root index j = 0 .. T - 1:
    root_exp = FIRST_ROOT + ((FIRST_ROOT + 1) % 2) + 2*j
    ROOT     = alpha ^ root_exp
    r_s      = 0

    on every input beat:
        w_chain[0] = r_s
        for u = 0 .. B - 1:
            symbol     = in_data[u] embedded as GF element {0..0, bit}
            w_chain[u+1] = w_chain[u] * ROOT ^ symbol
        w_next = w_chain[B]
        for u = 1 .. B:
            if w_in_count == u:
                w_next = w_chain[u]
        r_s <= w_next

    out_syndromes[j] = r_s

out_no_error = (out_syndromes == 0)
```

In the RTL. The syndrome fub is `rtl/fub/bch_syndrome_unit.sv:50`; it instantiates `T_BITS` copies of `gf_syndrome_cell` (`:128-144`) and computes `out_no_error` from the packed syndrome vector (`:155`). The shared `gf_syndrome_cell` itself is one `gf_mul_const` and a register (`projects/components/ecc-ip/reed-solomon/rtl/fub/gf/gf_syndrome_cell.sv:24-32`). Don't go looking for S_2 or S_4 in this block—they're derived in the KES.

29.3 Key-equation solver (bch_key_equation_solver)

The solver turns the syndromes into the error-locator polynomial $\Lambda(x)$. For a binary BCH code only $\Lambda(x)$ is needed; there's no Forney stage because every error magnitude is 1.

The math. The key equation is

$$\Lambda(x)\cdot S(x)\equiv\Omega(x)\,(\,mod\,x^{2t})$$

where $S(x)=S_1+S_2x+\ldots+S_{2t}x^{2t-1}$. The reformulated inversionless Berlekamp-Massey (riBM) algorithm solves it in exactly $2t$ iterations. One iteration uses the discrepancy $\delta=\Delta_0$ and a swap decision:

$$\Delta_i(r+1)=\gamma(r)\cdot\Delta_{i+1}(r)\oplus\delta(r)\cdot\Theta_i(r)$$

$$\Theta_i(r+1)=\text{swap}\,?\,\Delta_{i+1}(r):\Theta_i(r)$$

$$\gamma(r+1)=\text{swap}\,?\,\delta(r):\gamma(r)$$

$$k(r+1)=\text{swap}\,?\,-k(r)-1:k(r)+1$$

The pseudocode. The syndrome expander reconstructs $S_1\ldots S_{2t}$ from the t odd input lanes; note that this expander lives in `bch_key_equation_solver`, not in the syndrome unit.

```
// syndrome expander (inside bch_key_equation_solver)
for i = 1 .. 2*t:
    if i is odd:
        w_full[i-1] = in_syndromes[(i-1)/2]
    else:
        w_full[i-1] = w_full[i/2 - 1] ^ 2   // S_{2j} = S_j^2

// riBM array (shared from the reed-solomon library)
NPE = 3*t + 1
for i = 0 .. NPE - 1:
    if i < 2*t:      init[i] = w_full[i]
    elif i == 3*t:   init[i] = 1
    else:            init[i] = 0
    Delta[i] = init[i]
    Theta[i] = init[i]

gamma = 1
k     = 0
for iter = 1 .. 2*t:
    delta = Delta[0]
    swap  = (delta != 0) && (k >= 0)
    for i = 0 .. NPE - 1:
        Delta_next[i] = gamma * Delta[i+1] ^ delta * Theta[i]
        Theta_next[i] = swap ? Delta[i+1] : Theta[i]
```

```
    Delta = Delta_next
    Theta = Theta_next
    gamma = swap ? delta : gamma
    k     = swap ? -k-1 : k+1

// BCH wrapper readout
for i = 0 .. T:
    out_lambda[i] = Delta[t + i]
out_lambda_degree = degree of Delta[t .. 3*t]
out_more_than_t   = any Delta[t+i] != 0 for i > T
```

In the RTL. The BCH wrapper reconstructs the full syndrome sequence at `rtl/fub/bch_key_equation_solver.sv:109-134`, instantiates the imported `key_equation_solver_ribm` at `:150-167`, and trims the locator to `Lambda_0 .. Lambda_t` at `:180-185`. The riBM module itself is in the reed-solomon component tree (`projects/components/ecc-ip/reed-solomon/rtl/fub/key_equation_solver_ribm.sv:95,137-145` for initialization, `:180-188` for the control update, and `:196-213` for output readout). A sister description of the same riBM math lives at `../../../../reed-solomon/docs/reed_solomon_has/ch07_understanding_the_math/05_every_stage_math_to_pseudocode.md`. The per-cell update is in `projects/components/ecc-ip/reed-solomon/rtl/fub/gf/ribm_pe.sv:24-34,62-83`. One hard constraint: `KES_ALGO` isn't really a choice today—anything other than "RIBM" errors out at elaboration (`rtl/fub/bch_key_equation_solver.sv:103-105`).

29.4 Chien search (bch_chien_search)

The Chien search evaluates $\Lambda(x)$ at every position and flags the zeros.

The math. Position p (counted from the first transmitted bit) has locator value $X_p = \alpha^{n-1-p}$. The position is in error when

$$\Lambda\left(X_p^{-1}\right) = 0, X_p^{-1} = \alpha^{-(n-1-p)}$$

Within one beat the lane offset u evaluates position $p+u$:

$$\Lambda\left(X_{p+u}^{-1}\right) = \sum_{i=0}^{t} \Lambda_i \cdot \alpha^{-i(n-1-p)} \cdot \alpha^{iu}$$

The pseudocode. `r_c[i]` holds the coefficient contribution for the first position of the current beat; `LANE_K[u,i]` is the constant α^{iu}.

```
for i = 0 .. T:
    r_c[i] = in_lambda[i] * alpha^{-i*(N-1)}

p = 0
while p < N:
    for u = 0 .. B - 1:
        sum = 0
        for i = 0 .. T:
            sum ^= LANE_K[u,i] * r_c[i]
        w_root[u] = (sum == 0)

    out_flip_en    = w_root & position_valid(p .. p+B-1)
    out_root_count += count(w_root & position_valid)

    p += B
    for i = 0 .. T:
        r_c[i] *= alpha^{i*B}
```

In the RTL. The coefficient cells are initialized and stepped at `rtl/fub/bch_chien_search.sv:135-156`; the per-beat lane sum and root detection are at `:174-208`; the running root count is at `:219-231` and `out_flip_en` at `:210-214`. Because the code is binary, a root is a pure bit-flip location and no Forney stage is needed (`rtl/fub/bch_chien_search.sv:9-11`).

29.5 Verdict and correction (bch_decoder_core)

The decoder core collects the Chien results, decides whether the block is correctable, and applies the flips.

The math. A block is correctable when

$$\text{degree}(\Lambda) \le t \text{ and root count} = \text{degree}(\Lambda)$$

and, when enabled, the corrected stream re-check syndromes are all zero. For a binary BCH code every error value is 1, so correction is

$$\hat{c}_p = R_p \oplus e_p$$

where $e_p = 1$ for every found root position p. An uncorrectable block passes through unchanged.

The pseudocode.

```
// during Chien: capture beat data and flip mask
r_buf[beat]  <= in_data
r_flip[beat] <= w_chien_out_flip_en

// final verdict
w_verdict_correctable = !r_more_than_t
                      && (r_chien_root_count == r_lambda_degree)
                      && (!ENABLE_RECHECK || w_rechk_out_no_error)

// release
r_release_apply = w_verdict_correctable
out_data = r_buf[idx] ^ (r_flip[idx] & {B{r_release_apply}})

// frame-err or uncorrectable: r_release_apply = 0, block emitted
unchanged
```

In the RTL. The block buffer and flip-mask array are at `rtl/macro/bch_decoder_core.sv:148-150`; the verdict expression is at `:359-365`; the XOR-based flip application is at `:377-380`; the capture of `r_flip` is at `:520-524`; and the uncorrectable/frame-err pass-through paths are at `:535-547,558-569`. The trap here is subtle: if the verdict is "uncorrectable," `r_release_apply` stays zero and the block is emitted bit-for-bit unchanged—don't assume the corrector always XORs something.

29.6 Trace

The Python script `ch07_math_trace.py` in this directory reproduces the section-7.3 example with the same field, the same codeword, and the same errors. Section 7.3 now states its convention explicitly: "bit j" means polynomial exponent j, and the RTL reports the complementary transmission-order position $p = n - 1 - j$. The script's output is:

```
BCH(15,7) t=2 trace (bit 14 first):
  codeword           = 100110111000010
  received           = 100100111001010
  S_1                = alpha^12  (1111)
  S_3                = alpha^7   (1011)
  S_2                = alpha^9   (1010)
  S_4                = alpha^3   (1000)
  raw riBM Lambda    = alpha^1, alpha^13, alpha^14
  monic locator      = alpha^13 + alpha^12*X + X^2
  Chien roots (exp)  = [3, 10]
  RTL positions p    = [4, 11]
  corrected          = 100110111000010
```

Table 7.12: Numeric trace of the section-7.3 example

Quantity	Value	Note
S_1	α^{12}	matches Table 7.7
S_3	α^{7}	matches Table 7.7
raw riBM $\Lambda_0, \Lambda_1, \Lambda_2$	$\alpha^{1}, \alpha^{13}, \alpha^{14}$	RTL array output (BCH wrapper keeps first three lanes)
monic locator	$X^2 + \alpha^{12} X + \alpha^{13}$	reciprocal-scaled form used by the section-7.3 Chien table
Chien roots (exponents)	$\{3, 10\}$	the flipped bits from section 7.3
RTL positions p	$\{4, 11\}$	transmission-order indices; see section 7.3 for the convention
corrected word	`100110111000010`	identical to the section-7.2

Quantity	Value	Note
		codeword

The raw riBM array output is a scalar multiple of the reciprocal of the textbook monic locator. Evaluating that raw polynomial at $\alpha^{-(n-1-p)}$ is why the RTL position counter reports $p=4$ and $p=11$ for the same two errors that section 7.3 labels as exponents 10 and 3.